

Table of Contents

Early Claim and Lapsed Prediction	1
1. Business Understanding	1
1.1 Problem Statements	1
1.2 Objectives	1
2. Data Understanding	1
2.1 Features Breakdown	2
2.1.1 MiCare Claim	2
2.1.2 Traditional	5
2.1.3 Investment Linked	11
2.1.4 Customer	17
2.2 Merge Datasets	18
3. Exploratory Data Analysis (EDA)	19
3.1 Drop Duplicate POLICY_NO	19
3.2 Fill in the blank (RACE)	20
3.3 Handling Outliers	22
3.4 Data Transformation	25
3.5 Visualization	27
3.5.1 Categorical Features	27
3.5.2 Numeric/ Continuous Features	28
3.5.3 Target Feature	30
4. Data Preparation	31
4.1 Clean Whitespaces	31
4.2 Drop Duplicate POLICY_NO	32
4.3 Fill in the Blank (RACE)	34
4.4 Create Claim Indicator	35
4.5 Handling Outliers	36
4.6 Feature Encoding	38
4.7 Train Test Split	42
5. Modeling	45
5.1 Handle Imbalanced Dataset	46
5.2 Hyperparameter Tuning	46
5.3 Model Evaluation	48
5.3.1 Tuned Parameters	48
5.3.2 Result for RESAMPLED Data	49
5.3.3 Result for Whole Data	54

5.3.4 Find the Suitable Threshold	58
5.4 Feature Evaluation	59
6. Save Model	61
7. Test Model	61
8. Deployment	62
8.1 Streamlit	62
8.1.1 Installation Steps & Requirements	63
8.1.2 App.py	63
8.1.3 DataManipulation.py	69
8.2 Docker	72
8.2.1 Docker Web Application Installation Steps	72
8.2.2 Docker Terminologies	73
8.2.3 Dockerfile	74

Early Claim and Lapsed Prediction

1. Business Understanding

Tokio Marine Life Insurance Malaysia (TMLM) was looking for a way to help them automate the process of identifying which insurance claims are early claimed. Previously, this process was done manually by humans, which resulted in slow processing speed and erroneous indirectly as erroneous increases when people were exhausted. So, TMLM has collaborated with Tunku Abdul Rahman University (TARUC) as well as Dynafront company to involve in this project to build a model that can help them to classify which insurance claims are early claimed and which are not so that the process of identifying which insurance claims are early claimed can be boosted which can lead to higher productivity. Throughout this project, TARUC will provide domain knowledge of data science, machine learning and Artificial Intelligence, while TMLM will provide us with data and insurance claims knowledge.

1.1 Problem Statements

1. The LightGBM model is not re-trainable as the code cannot produce the same result.
2. The data used to train the LightGBM model is not accurate because of errors made during the merging and mapping processes.
3. New Requirement: The model should provide highly precise predictions rather than high recall, which leads to too many false positives.

1.2 Objectives

1. Create a new re-trainable model.
2. Re-run the data merging and mapping processes.
3. Optimize the model to achieve the highest level of precision.

2. Data Understanding

We have four datasets:

- **MiCare (v2_masked)**: Medical claims from policyholders.
- **Trad (v2_masked)**: Details of Traditional Life Insurance.
- **IL (v2_masked)**: Details of Investment-Linked.

- **Customer (masked):** Customer Details.

2.1 Features Breakdown

2.1.1 MiCare Claim

Variable Name	Description
Year	Year of claim
Month	Month of claim
POLICY_NO	Policy Number
ClaimType	Claim type (Cashless/Reimbursement)
DOA	Date of Admission
DOD	Date of Discharge
RBRCExpPay	Daily Room & Board (Room Expenses)
RBFMEExpPay	Daily Room & Board (Food/Meals/GST Expenses)
ICURCExpPay	ICU Room Expenses
SASurgeonExpPay	Surgeon Expenses
SAAnaExpPay	Anesthetic Expenses
OperatingExpPay	Operating Theater Expenses
HSSExpPayTotal	Total Expenses of Hospital Services & Supplies
HABNCExpPay	Nursing Care / Procedure / Treatment Expenses
HABMedExpPay	Medicine / Pharmacy / Injection / HomeDrugs Expenses
HABLabExpPay	Laboratory / Diagnostic / X-ray / Scan / Imaging Expenses
HSSTherapyExpPay	Therapy / Physiotherapy Expenses
HSSExpPay	Medical Supplies / Consumables Expenses
HSSOtherExpPay	Hospital Services & Supplies / Others Expenses
PhysicianExpPay	In-Hospital Physician Visit Expenses

LodgerExpPay	Lodger Expenses
AMBExpPay	Ambulance Expenses
MedRptExpPay	Medical Report Expenses
DSSurgExpPay	Surgical Expenses
DSHCExpPay	Hospital Expenses
PABDiagExpPay	Pre-Hospital Diagnostic Tests Expenses
PABSCFSExpPay	Pre-Hospital Specialist Consultation Expenses
PHTExpPay	Post Hospitalization Treatment
AccOPExpPay	Emergency Accidental Outpatient Treatment Expenses
AccDenExpPay	Emergency Accidental Dental Treatment Expenses
PABSCFSecExpPay	Second Surgical Opinion Expenses
MajorExpPay	Major Surgical Bonus Expenses
OPPhyExpPay	Outpatient Physiotherapy Treatment Expenses
MthOPCancerExpPay	Monthly Out-patient Kidney Cancer Treatment Expenses
OPCancerExpPay	Annual Out-patient Kidney Cancer Treatment Expenses
MthOPKidExpPay	Monthly Out-patient Kidney Dialysis Treatment Expenses
LifeOPKidExpPay	Annual Out-patient Kidney Dialysis Treatment Expenses
OrganExpPay	Organ Transplantation Dialysis Expenses
IPPhyExpPay	In-Patient Treatment for Mental Illness Expenses
PhysioExpPay	In-Hospitalization Physiotherapy Treatment Expenses
ImplantExpPay	Surgical Implant Expenses
DailyCashExpPay	Daily Cash Allowance at Government Hospital
ESTExpPay	Emergency Sickness Treatment Expenses
HNCTIExpPay	Home Nursing Care Expenses due to Terminal Illness
MaternityExpPay	Maternity Benefit

DeaBenExpPay	Death Benefit
AccDeaBenExpPay	Accidental Benefit
FuneExpPay	Funeral Expenses
DreadExpPay	Dread Disease Benefit / Max Limit for GH
PerDisExpPay	Total and Permanent Disability Expenses
TotalCopay	Total Co-Payment (Total medical expenses that customers should bear as their insurance policy doesn't fully bear the costs)
RBStay	Days of Room and Boards Stay
ICUStay	Days of ICU Stay
Gender	Sex
InceptionDate	Inception Date (Insurans effective start date)
Age	Age
HospitalCode	Hospital Code (Each different code represents a hospital)
DiagCode	Diagnosis Code (ICD code)
DiagDesc	Diagnosis Description
ClaimNo	Claims Number
ClaimIssuedDT	Claims Issued Date
DoctorCode	Doctor Code (Each different code represents a doctor)
Incured Amount	Incurred Amount
InsuredAmt	Insured Amount
excess paid by pt	Formula: Incurred Amount - Insured Amount May include the cost of co-payment as well as some medical costs that the insurance policy doesn't bear
DiagProcedure	Diagnosis Procedure (details)
Type	Ordinary Life (Traditional) / Investment-linked

2.1.2 Traditional

Variable Name	Description	Value
POLICY_NO	Policy number	Primary key
BRANCH_CODE	Branch code	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu', 'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor' 'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'
POLICY_STATUS	Policy status	
CURRENCY_CODE	Currency code	RM: Ringgit Malaysia
RISK_CODE	Component code	Every type of insurance has its own risk code.
LIFE_COVERAGE	Life Coverage	1 - Yes 0 - No
COVER_NO	Number of Coverage	Primary key
RIDER_NO	Number of Rider	Primary key

RISK_TERM	Term of basic plan	The period of time that the insured is covered. (Year)
RISK_STATUS	Risk status	I - Inforce T - Terminate
POL_ISSUE_YEAR	Year of policy issue date	Year
POL_COMMENCE_DATE	Policy commencement date (Can treat as policy start date)	Date
SEX	Gender of the first life insured	M - Male F - Female U - Unknown
OCCUPATION_CLASS	Occupation class	A, B, C, D
RACE	Race code of the first life assured	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
MARITAL_STATUS	Marital status of the first life assured	D - Divorced M - Married N - Not Disclosed S - Single W - Widowed X - Unknown
ENTRY_AGE	The age at which the customer owned the policy	Age
SMOKER	Smoker / Non-Smoker status	S - Smoker N - Not-smoker U - Unknown
PAYMENT_MODE	Code of Payment Mode	1 - Yearly 2 - Half-yearly 4 - Quarterly 12 - Monthly
RSK_SUM_ASSURE	Risk Sum Assured	0, 100, 150, 200, 300, 400,

	It is the Amount of Risk that the company takes on your life. It is the amount assured on your death (Sum Assured). A sum assured is a fixed amount that is paid to the policy holder in the unfortunate event of the policyholder's demise. The insurance company pays this money as per the sum chosen by you at the time of purchasing the policy.	600
ANUAL_PREMIUM	Annualized premium (exclude extra premium)	Amount to be paid
ANUAL_EX_PREMIUM	Annualized extra premium	Amount to be paid
MATURE_AGE	Risk cessation age / maturity age of the policy	Age
PAYMNT_TERM	Payment term of basic plan How long (in years) the customer wants to pay for his/her policy	Year
SUBSTD_FLAG	Substandard flag of the first life assured	N - No Y - Yes
EXP_DATE	Expiry Date/ Risk Cessation Date	Date
RSK_COMMENCE_DATE	Risk commencement date	Date
RSK_STATUS_DATE	Last date of risk status changed	Date
PAR_INDICATE	Par / Non-Par Indicator	Indicator
POL_LAPSE_DATE	Policy lapse date either from history or potential lapse date	Date
POL_STATUS_DATE	Last date of policy status changed	Date
POLICY_YEAR	Year of policy	Year

EXTRA_LOAD	Extra Loading in percentage (%) It is an amount of additional loading that Tokio Marine imposes on the customers for the higher risk customers like customers who have high blood pressure, pre-existing conditions like diabetes, etc.	Percentage
POL HOLDER_TYPE	Channel	
SERVICE_AGENT_NO	Servicing agent number	Primary Key
SERVICE_AGENT_SEX	Servicing agent gender	M - Male F - Female U - Unknown
SERVICE_AGENT_RACE	Servicing agent race	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
SERVICE_AGENT_BRAN CH_CODE	Servicing agent branch code	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu', 'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor'

		'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'
SERVICE_AGENT_EDU_LEVEL	Servicing agent educational level	S - Secondary U - Degree P - Primary H - A-level T - Diploma
SERVICE_AGENT_AGE	Servicing agent age	Age
SERVICE_AGENT_JOIN_DATE	Servicing agent join date	Date
SERVICE_AGENT_OCCUPATION_CODE	Servicing agent occupation code	A, B, C, D
SERVICE_AGENT_POSTCODE	Servicing agent postcode	Postcode
PREM_STATUS	Premium status	
LAST_PAYMT_DATE	Last payment date	Date
FIRST_ISSUE_DATE	First issue date	Date
POL_ISSUE_DATE	Policy issue date	Date
PAID_TO_DATE	Date of last premium payment	Date
CLAIM_RECOD_DATE	Claims Coverage Start Date	Date
PAYMENT_METHOD	Payment method	B - BIRO Billing C - Direct Selling (Cash) D - Direct Debit G - Group Billing N - No Billing R - Credit Card S - CPF-SA Billing F- e-Mandate K - Police Koperasi

SELL_AGENT_NO	Selling agent number	Primary Key
SELL_AGENT_SEX	Selling agent gender	F - Female M - Male
SELL_AGENT_RACE	Selling agent race	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
SELL_AGENT_EDU_LEVEL	Sell agent educational level	S - Secondary U - Degree P - Primary H - A-level T - Diploma
SELL_AGENT_AGE	Sell agent age	Age
SELL_AGENT_JOIN_DATE	Sell agent join date	Date
SELL_AGENT_OCCUPATION_CODE	Sell agent occupation code	A, B, C, D
SELL_AGENT_POSTCODE	Sell agent postcode	Postcode
SELL_AGENT_BRANCH_CODE	Selling agent branch code (Original agent branch code)	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu',

		'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor' 'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'
FLAT_RATE_LOAD	Flat Rate Loading	
FLAT_RATE_LOAD_YEAR	Flat rate loading year	Year

2.1.3 Investment Linked

Variable Name	Description	Value
POLICY_NO	Policy number	Primary Key
BRANCH_CODE	Branch code	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu', 'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor' 'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'

POLICY_STATUS	Policy status	
CURRENCY_CODE	Currency code	RM - Ringgit Malaysia
RISK_CODE	Component code	Every type of insurance has its own risk code.
LIFE_COVERAGE	Life Coverage	
COVER_NO	Number of Coverage	Primary Key
RIDER_NO	Number of Rider	Primary Key
RISK_TERM	Term of basic plan	The period of time that the insured is covered. (Year)
RISK_STATUS	Risk status	I - Inforce T - Terminate
POL_ISSUE_YEAR	Year of policy issue date	Year
POL_COMMENCE_DATE	Policy commencement date	Date
SEX	Gender of the first life insured	F - Female M - Male U - Unknown
OCCUPATION_CLASS	Occupation class (A, B, C, D)	A, B, C, D
RACE	Race code of the first life assured	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
MARITAL_STATUS	Marital status of the first life assured	D - Divorced M - Married N - Not Disclosed S - Single W - Widowed X - Unknown
ENTRY_AGE	Entry age for component code	Age

SMOKER	Smoker / Non-Smoker status	S - Smoker N - Not-Smoker U - Unknown
PAYMENT_MODE	Code of Payment Mode	1 - Yearly 2 - Half-yearly 4 - Quarterly 12 - Monthly
RSK_SUM_ASSURE	Risk Sum Assured It is the Amount of Risk that the company takes on your life. It is the amount assured on your death (Sum Assured). A sum assured is a fixed amount that is paid to the policy holder in the unfortunate event of the policyholder's demise. The insurance company pays this money as per the sum chosen by you at the time of purchasing the policy.	0, 100, 150, 200, 300, 400, 600
SINGLE_PREMIUM	Single premium	Amount to be paid
MATURE_AGE	Risk cessation age / maturity age	Age
PAYMNT_TERM	Payment term of basic plan How long (in years) the customer wants to pay for his/her policy	Year
SUBSTD_FLAG	Substandard flag of the first life assured	N - No Y - Yes
EXP_DATE	Expiry Date/ Risk Cessation Date	Date
RSK_COMMENCE_DATE	Risk commencement date	Date
RSK_STATUS_DATE	Last date of risk status changed	Date
POL_LAPSE_DATE	Policy lapse date either from history or	Date

	potential lapse date	
POL_STATUS_DATE	Last date of policy status changed	Date
POLICY_YEAR	Year of policy	Year
TOPUP_PREMIUM	Top up Premium	Amount to be paid
EXCESS_PREMIUM	Excess Premium	Amount to be paid
EXTRA_LOAD	Extra Loading in percentage (%) It is an amount of additional loading that Tokio Marine imposes on the customers for the higher risk customers like customers who have high blood pressure, pre-existing conditions like diabetes, etc.	Percentage
SERV_BRANCH_CODE	Servicing agent branch code	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu', 'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor' 'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'
POL HOLDER_TYPE	Channel	

SERVICE_AGENT_NO	Servicing Agent number	Primary Key
SERVICE_AGENT_BRANCH_CODE	Agents branch code	Primary Key
SERVICE_AGENT_SEX	Servicing agent gender	F - Female M - Male
SERVICE_AGENT_RACE	Servicing agent race	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
SERVICE_AGENT_EDUCATIONAL_LEVEL	Servicing agent educational level	S - Secondary U - Degree P - Primary H - A-level T - Diploma
SERVICE_AGENT_AGE	Servicing agent age	Age
SERVICE_AGENT_JOIN_DATE	Servicing agent join date	Date
SERVICE_AGENT_OCCUPATION_CODE	Servicing agent occupation code	A, B, C, D
SERVICE_AGENT_POSTCODE	Servicing agent postcode	Postcode
RISK_STATUS	Risk status	I - Inforce T - Terminated
PREM_STATUS	Premium status	
LAST_PAYMT_DATE	Last payment date	Date
POL_ISSUE_DATE	Policy issue date	Date
INDCATE_FLEX_PREM	Indicator for Flex Premium	Indicator
PAID_TO_DATE	Date of last premium payment	Date

RSK_COMMENCE_DATE	Risk commencement date	Date
PAYMENT_METHOD	Payment method	B - BIRO Billing C - Direct Selling (Cash) D - Direct Debit G - Group Billing N - No Billing R - Credit Card S - CPF-SA Billing F- e-Mandate K - Police Koperasi
SELL_AGENT_NO	Selling agent number	Primary Key
SELL_AGENT_BRANCH_CODE	Selling agent branch code (Original agent branch code)	'J1': 'JohorBharu' 'P2': 'Butterworth' 'A1': 'Ipoh' 'N1': 'Seremban', 'M1': 'Melacca' 'P1': 'Penang' 'W1': 'KualaLumpur' 'S2': 'Miri' 'S1': 'Kuching' 'B1': 'Klang', 'K1': 'AlorStar' 'D1': 'KotaBharu' 'C1': 'Kuantan' 'S3': 'Sibu', 'H2': 'KotaKinabalu' 'H1': 'Sandakan' 'KE': 'FinancialAdvisor' 'FA': 'FinancialAdvisor' 'RH': 'RHB' 'HO': 'RHB' Others: 'RHB'
SELL_AGENT_SEX	Selling agent gender	F - Female M - Male U - Unknown

SELL_AGENT_RACE	Selling agent race	BN - Bumiputera CH - Chinese IN - Indian MA - Malay OT - Other
SELL_AGENT_EDU_LEVEL	Sell agent educational level	S - Secondary U - Degree P - Primary H - A-level T - Diploma
SELL_AGENT_AGE	Sell agent age	Age
SELL_AGENT_JOIN_DATE	Sell agent join date	Date
SELL_AGENT_OCCUPATION_CODE	Sell agent occupation code	A, B, C, D
SELL_AGENT_POSTCODE	Sell agent postcode	Postcode
FACULATIVE_IND	Facultative Indicator	N: No

2.1.4 Customer

Variable Name	Description
POLICY_NO	Policy number
CLIENT_NO	Client number
POSTCODE	Postcode
STATE	State
WEIGHT	Weight
HEIGHT	Height
BMI	Body Mass Index

2.2 Merge Datasets

```
# Read MiCare dataset
```

```
df_micare = pd.read_excel(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw Data\Archive\2018-2021\MiCare (v2_masked).xlsx')
```

```
# Read Trad dataset
```

```
df_trad = pd.read_excel(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw Data\Archive\2018-2021\Trad (v2_masked).xlsx')
```

```
# Read IL dataset
```

```
df_IL = pd.read_excel(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw Data\Archive\2018-2021\IL (v2_masked).xlsx')
```

```
# Read customer dataset
```

```
df_cust = pd.read_excel(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw Data\Archive\2018-2021\Customer (masked).xlsx')
```

```
df_trad['Type'] = ['OL'] * df_trad.shape[0]
```

```
df_IL['Type'] = ['IL'] * df_IL.shape[0]
```

```
policy_df = pd.concat([df_trad, df_IL], axis=0) # Concatenate Trad and IL first
```

```
policy_df.sort_values(by=['POLICY_NO'], inplace=True)
```

```
policy_df.reset_index(inplace=True, drop=True)
```

```
# Drop again the print(duplicate rows and keep the last appearance
```

```
print(policy_df.shape)
```

```
policy_df.drop_duplicates(keep='last', inplace = True)
```

```
print(policy_df.shape)
```

```
claim_df_2018 = policy_df.merge(df_cust[['POLICY_NO', 'STATE', 'BMI']], on='POLICY_NO', how='left') # Then merge with Customer
```

```
claim_df_2018 = claim_df_2018.merge(df_micare[['POLICY_NO', 'DOA',
'EarlyClaimAndLapsed']], on='POLICY_NO', how='left') # Then merge with MiCare
claim_df_2018.sort_values(by=['POLICY_NO'], inplace=True)
claim_df_2018.reset_index(inplace=True, drop=True)
claim_df_2018.head(10)
```

```
claim_df_2018 = claim_df_2018.dropna(subset=['EarlyClaimAndLapsed'])
claim_df_2018.head()
```

```
claim_df_2018.to_excel(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw
Data\2018-2021_dataset_new.xlsx', index=False)
claim_df_2018.to_csv(r'\\10.188.78.123\CP_Shared\Early Claims Then Lapsed\Raw
Data\2018-2021_dataset_new.csv', index=False)
```

3. Exploratory Data Analysis (EDA)

```
# Read Cleaned Whitespaces Dataset
df1 = pd.read_csv(r'2018-2021_dataset_new_cleaned_whitespace.csv', index_col=0)
```

3.1 Drop Duplicate POLICY_NO

```
print('Before:', df1[df1[['POLICY_NO']].duplicated()].shape[0])
df1 = df1.sort_values('EarlyClaimAndLapsed',
ascending=False).drop_duplicates(subset=['POLICY_NO'],
keep='first').sort_index()
print('After: ', df1[df1[['POLICY_NO']].duplicated()].shape[0])
```

Output:

Before: 254177

After: 0

```
# Select part of the policy holder related features & sell agent features
# Purpose: Present to Mr Toi
df_eda = df1[['POLICY_NO', 'RACE', 'SEX', 'SMOKER', 'BMI', 'OCCUPATION_CLASS',
'MARITAL_STATUS', 'STATE', 'RISK_TERM', 'RSK_SUM_ASSURE', 'PAYMENT_MODE',
```

```

        'SELL_AGENT_NO', 'SELL_AGENT_BRANCH_CODE', 'SELL_AGENT_SEX',
        'SELL_AGENT_RACE', 'SELL_AGENT_EDU_LEVEL', 'SELL_AGENT_AGE',
        'SELL_AGENT_JOIN_DATE', 'SELL_AGENT_OCCUPATION_CODE', 'SELL_AGENT_POSTCODE',
        'EarlyClaimAndLapsed']]
df_eda

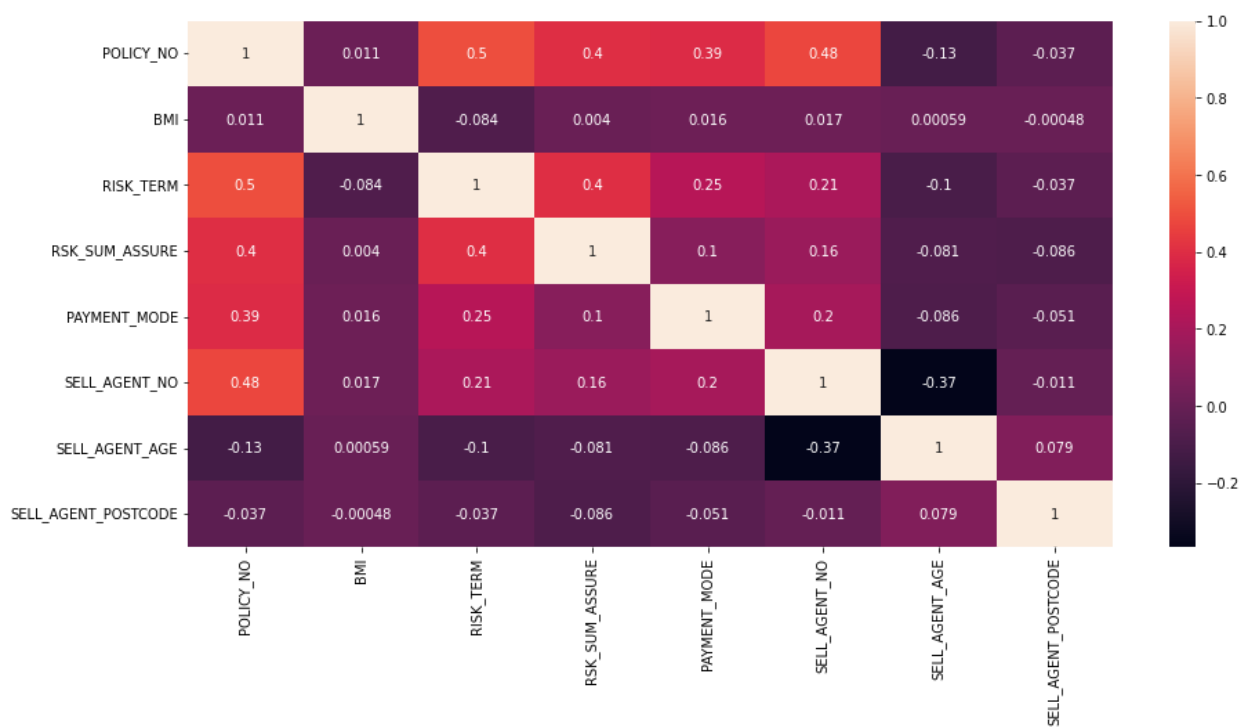
```

```

#plot the correlation using heatmap
plt.figure(figsize=(15,7))
sns.heatmap(df_eda.corr(), annot=True)

```

Output:



3.2 Fill in the blank (RACE)

Example

BEFORE

ROW	POLICY_NO	RACE
1	1234	CH
2	1234	
3	1234	

4	1234	CH
AFTER		
ROW	POLICY_NO	RACE
1	1234	CH
2	1234	CH
3	1234	CH
4	1234	CH

```
df_eda.isnull().sum()
```

Output:

```
POLICY_NO      0
RACE           6523
SEX            0
SMOKER         42
BMI            151
OCCUPATION_CLASS 17
MARITAL_STATUS 0
STATE          7
RISK_TERM      0
RSK_SUM_ASSURE 0
PAYMENT_MODE   0
SELL_AGENT_NO  0
SELL_AGENT_BRANCH_CODE 0
SELL_AGENT_SEX 125
SELL_AGENT_RACE 133
SELL_AGENT_EDU_LEVEL 2766
SELL_AGENT_AGE 125
SELL_AGENT_JOIN_DATE 0
SELL_AGENT_OCCUPATION_CODE 125
SELL_AGENT_POSTCODE 6
EarlyClaimAndLapsed 0
dtype: int64
```

```
df_full = pd.read_csv('2018-2021_dataset_new (whole_data).csv')
```

```
for i in df_eda['POLICY_NO'].loc[(df_eda['RACE'].isnull() == True)].unique():

    if len(df_full['RACE'].loc[(df_full['POLICY_NO'] == i)].value_counts())
```

```
!=0:
    replace_val = df_full['RACE'].loc[(df_full['POLICY_NO'] ==
i)].value_counts().keys()[0]
    df_eda['RACE'].loc[(df_eda['POLICY_NO'] == i)] = replace_val
```

```
df_eda.isnull().sum()
```

Output:

```
POLICY_NO      0
RACE           13
SEX            0
SMOKER         42
BMI           151
OCCUPATION_CLASS 17
MARITAL_STATUS 0
STATE          7
RISK_TERM      0
RSK_SUM_ASSURE 0
PAYMENT_MODE   0
SELL_AGENT_NO  0
SELL_AGENT_BRANCH_CODE 0
SELL_AGENT_SEX 125
SELL_AGENT_RACE 133
SELL_AGENT_EDU_LEVEL 2766
SELL_AGENT_AGE 125
SELL_AGENT_JOIN_DATE 0
SELL_AGENT_OCCUPATION_CODE 125
SELL_AGENT_POSTCODE 6
EarlyClaimAndLapsed 0
dtype: int64
```

```
df_eda = df_eda.dropna()
```

```
df_eda.shape
```

Output:

```
(24477, 21)
```

3.3 Handling Outliers

```

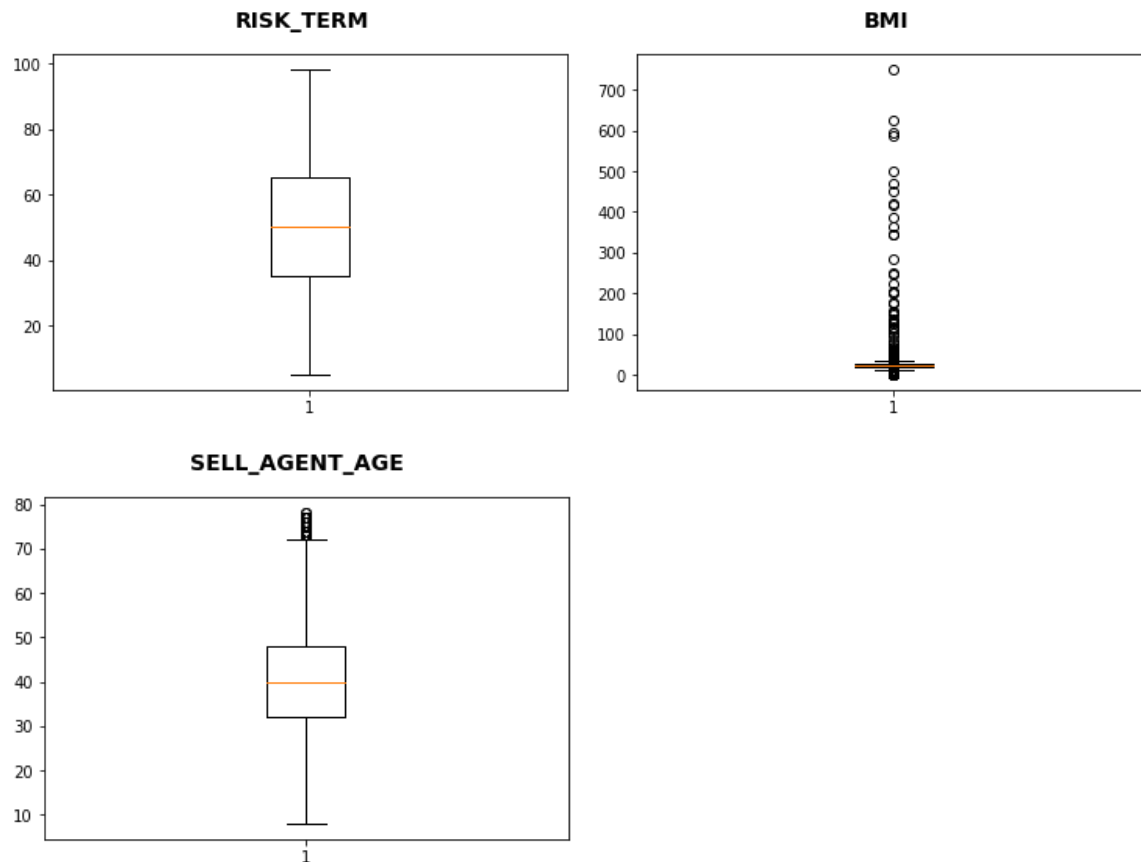
num_col = ['RISK_TERM', 'BMI', 'SELL_AGENT_AGE']

#Outliers detection
#Show data in boxplots
Numeric_dic = {}
for x in num_col:
    Numeric_dic[x]= df_eda[x].to_numpy()

for x in Numeric_dic:
    fig = plt.figure()
    fig.suptitle(x, fontsize=14, fontweight='bold')
    ax = fig.add_subplot(111)
    ax.boxplot(Numeric_dic[x])
    plt.show()

```

Output:



```

#Remove outliers

```

```

for x in num_col:
    Q1 = df_eda[x].quantile(0.25)
    Q3 = df_eda[x].quantile(0.75)
    IQR = Q3 - Q1
    max_range = (Q3+ (1.5*IQR))
    min_range = (Q1- (1.5*IQR))
    df_eda[x] = df_eda[x].loc[(df_eda[x] < max_range)]
    df_eda[x] = df_eda[x].loc[(df_eda[x] > min_range)]

#remove missing values again
df_eda = df_eda.dropna()

```

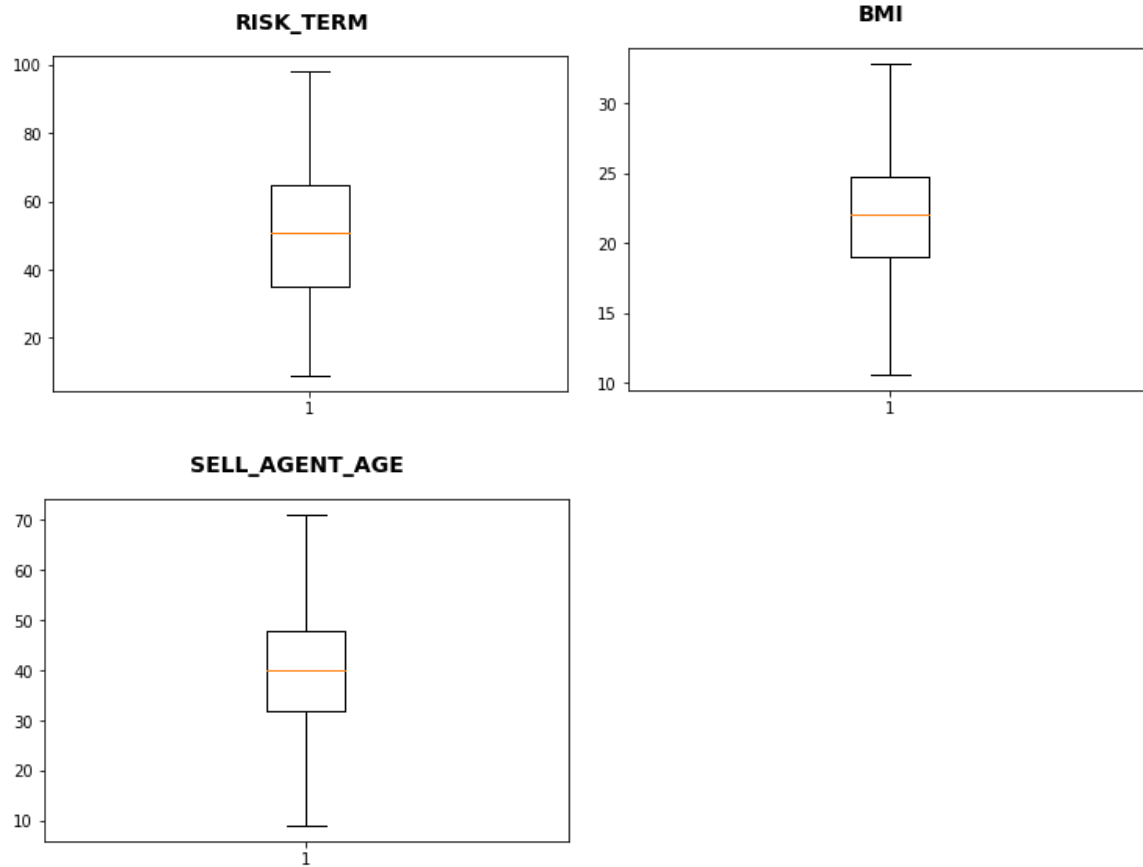
```

#Show data in boxplots
Numeric_dic = {}
for x in num_col:
    Numeric_dic[x]= df_eda[x].to_numpy()

for x in Numeric_dic:
    fig = plt.figure()
    fig.suptitle(x, fontsize=14, fontweight='bold')
    ax = fig.add_subplot(111)
    ax.boxplot(Numeric_dic[x])
    plt.show()

```

Output:



3.4 Data Transformation

```
df_eda.reset_index(inplace=True, drop=True)
```

```
def mapPostCode(claim_df):
    i = 0
    while i < claim_df.count():
        if claim_df[i] in range (50000,60001) :
            claim_df[i] = 'KUALA LUMPUR'
        elif str(claim_df[i]).startswith('62') :
            claim_df[i] = 'PUTRAJAYA'
        elif str(claim_df[i]).startswith('87') :
            claim_df[i] = 'LABUAN'
        elif claim_df[i] in range (1000,2801) :
```

```

        claim_df[i] = 'PERLIS'
    elif claim_df[i] in range (5000,9811) or claim_df[i] == 14290 or
claim_df[i] == 14390 or claim_df[i] == 34950:
        claim_df[i] = 'KEDAH'
    elif claim_df[i] in range (10000,14401) :
        claim_df[i] = 'PENANG'
    elif claim_df[i] in range (15000,18501) :
        claim_df[i] = 'KELANTAN'
    elif claim_df[i] in range (20000,24301) :
        claim_df[i] = 'TERENGGANU'
    elif claim_df[i] in range (25000,28801) or
str(claim_df[i]).startswith('39') or str(claim_df[i]).startswith('49') or
claim_df[i] == 69000:
        claim_df[i] = 'PAHANG'
    elif claim_df[i] in range (30000,36811) :
        claim_df[i] = 'PERAK'
    elif claim_df[i] in range (40000,48301) or claim_df[i] in range
(63000,63301) or claim_df[i] == 64000 or claim_df[i] in range (68000,68101):
        claim_df[i] = 'SELANGOR'
    elif claim_df[i] in range (70000,73501) :
        claim_df[i] = 'NEGERI SEMBILAN'
    elif claim_df[i] in range (75000,78301) :
        claim_df[i] = 'MELAKA'
    elif claim_df[i] in range (79000,86901) :
        claim_df[i] = 'JOHOR'
    elif claim_df[i] in range (88000,91301) :
        claim_df[i] = 'SABAH'
    elif claim_df[i] in range (93000,98851) :
        claim_df[i] = 'SARAWAK'
    i+=1

mapPostCode(df_eda['SELL_AGENT_POSTCODE'])
df_eda

```

```

df_eda = df_eda.rename({'SELL_AGENT_POSTCODE': 'SELL_AGENT_STATE'}, axis=1)

```

3.5 Visualization

3.5.1 Categorical Features

```
claim_df = df_eda
claim_df['PAYMENT_MODE'] = claim_df['PAYMENT_MODE'].replace({12: 'Monthly',
                                                             1: 'Yearly',
                                                             2: 'Half-yearly',
                                                             4:
'Quarterly'}).astype(str)
numBin_df = pd.DataFrame()
for num_col in claim_df.select_dtypes(include = ['object']).keys().tolist():
    numBin_df[f'{num_col}'] = claim_df[num_col]

numBin_df
```

```
for cat_col in numBin_df[['SEX',
'SMOKER', 'OCCUPATION_CLASS', 'MARITAL_STATUS', 'STATE',
'PAYMENT_MODE', 'SELL_AGENT_BRANCH_CODE', 'SELL_AGENT_SEX', 'SELL_AGENT_RACE', 'SEL
L_AGENT_EDU_LEVEL', 'SELL_AGENT_JOIN_DATE', 'SELL_AGENT_OCCUPATION_CODE', 'SELL_AG
ENT_STATE', 'EarlyClaimAndLapsed']].keys().tolist():

    if cat_col != 'EarlyClaimAndLapsed':
        df_g = numBin_df[[cat_col, 'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().reset_index()
        df_g['percentage'] = numBin_df[[cat_col,
'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().groupby(level=0).apply(lambda x: 100 * x /
float(x.sum())).values
        df_g.columns = [cat_col, 'EarlyClaimAndLapsed', 'Counts', 'Percentage']

        # If the EarlyClaimAndLapsed column contain 'Y' value in the first row
of the dataframe, add a 'N' value to the first row
        if df_g.iloc[0:1, :]['EarlyClaimAndLapsed'].tolist()[0] == 'Y':
            df_g.loc[-1] = [df_g.iloc[0:1, :][cat_col].tolist()[0], 'N', 0, 0,
df_g.iloc[0:1, :][f'{cat_col}_SORT'].tolist()[0]] # adding a row
            df_g.index = df_g.index + 1 # shifting index
```

```

df_g.sort_index(inplace=True)

fig = px.bar(df_g, x=cat_col, y=['Counts'],
color='EarlyClaimAndLapsed', text=df_g['Percentage'].apply(lambda x:
'{0:1.2f}%'.format(x)), barmode='group')
fig.update_layout(title_text=f'{cat_col} vs EarlyClaimAndLapsed',
title_x = 0.5, font_size=12)
fig.update_yaxes(title_text='Count')
fig.write_html(f"EDA_{cat_col}.html")
fig.show()

```

Output:

The charts are saved as HTML files.

3.5.2 Numeric/ Continuous Features

```

# This function is used to bin numeric features where each category each bin
contain 5 values
# E.g. 0-4, 5-9, and so on
def bin_numeric_cols(val):
    if val < 5:
        return '0-4'
    elif val < 10:
        return '5-9'
    elif val < 15:
        return '10-14'
    elif val < 20:
        return '15-19'
    elif val < 25:
        return '20-24'
    elif val < 30:
        return '25-29'
    elif val < 35:
        return '30-34'
    elif val < 40:
        return '35-39'
    elif val < 45:
        return '40-44'
    elif val < 50:

```

```

        return '45-49'
    elif val < 55:
        return '50-54'
    elif val < 60:
        return '55-59'
    elif val < 65:
        return '60-64'
    elif val < 70:
        return '65-69'
    elif val < 75:
        return '70-74'
    elif val < 80:
        return '75-79'
    elif val < 85:
        return '80-84'
    elif val < 90:
        return '85-89'
    elif val < 95:
        return '90-94'
    else:
        return '95-99'

```

```

claim_df = df_eda
numBin_df = pd.DataFrame()
for num_col in claim_df.select_dtypes(exclude = ['object']).keys().tolist():
    if (claim_df.select_dtypes(include=['number'])[num_col].max() < 100) and
(claim_df.select_dtypes(include=['number'])[num_col].min() >= 0):
        numBin_df[f'{num_col}'] = claim_df[num_col].apply(bin_numeric_cols)

numBin_df = numBin_df.merge(claim_df['EarlyClaimAndLapsed'].to_frame(),
left_index=True, right_index=True)
numBin_df

```

```

for cat_col in numBin_df[['SELL_AGENT_AGE', 'RISK_TERM', 'BMI']].keys().tolist():

```

```

if cat_col != 'EarlyClaimAndLapsed':
    df_g = numBin_df[[cat_col, 'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().reset_index()
    df_g['percentage'] = numBin_df[[cat_col,
'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().groupby(level=0).apply(lambda x: 100 * x /
float(x.sum())).values
    df_g.columns = [cat_col, 'EarlyClaimAndLapsed', 'Counts', 'Percentage']
    df_g[f'{cat_col}_SORT'] = df_g[cat_col].apply(lambda x:
int(x.split('-')[-1]))
    df_g = df_g.sort_values(by=[f'{cat_col}_SORT'],
ascending=True).reset_index(drop=True)

    # If the EarlyClaimAndLapsed column contain 'Y' value in the first row
of the dataframe, add a 'N' value to the first row
    if df_g.iloc[0:1, :]['EarlyClaimAndLapsed'].tolist()[0] == 'Y':
        df_g.loc[-1] = [df_g.iloc[0:1, :][cat_col].tolist()[0], 'N', 0, 0,
df_g.iloc[0:1, :][f'{cat_col}_SORT'].tolist()[0]] # adding a row
        df_g.index = df_g.index + 1 # shifting index
        df_g.sort_index(inplace=True)
        df_g.drop(labels=[f'{cat_col}_SORT'], axis=1, inplace=True)

    fig = px.bar(df_g, x=cat_col, y=['Counts'],
color='EarlyClaimAndLapsed', text=df_g['Percentage'].apply(lambda x:
'{0:1.2f}%'.format(x)), barmode='group')
    fig.update_layout(title_text=f'{cat_col} vs EarlyClaimAndLapsed',
title_x = 0.5, font_size=12)
    fig.update_yaxes(title_text='Count')
    fig.write_html(f"EDA_{cat_col}.html")
    fig.show()

```

Output:

The charts are saved as HTML files.

3.5.3 Target Feature

```

def autopct_format(values):
    def my_format(pct):
        total = sum(values)

```

```

        val = int(round(pct*total/100.0))
        return '{:.1f}%\n({v:d})'.format(pct, v=val)
    return my_format

```

```

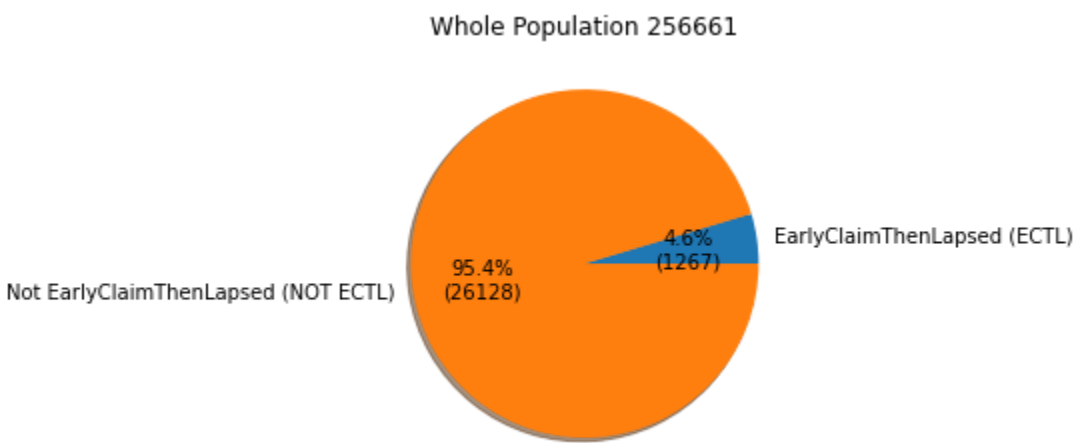
y = np.array([len(df_eda.loc[(df_eda['EarlyClaimAndLapsed'] == 'Y')]),
len(df_eda.loc[(df_eda['EarlyClaimAndLapsed'] == 'N')])])
mylabels = ["EarlyClaimThenLapsed (ECTL)", "Not EarlyClaimThenLapsed (NOT ECTL)"]

plt.title(f'Whole Population {df_eda.shape[0]}')
plt.pie(y, labels = mylabels, shadow = True, autopct=autopct_format(y))

plt.show()

```

Output:



4. Data Preparation

```

# Read Merged dataset
df1 = pd.read_csv('2018-2021_dataset_new.csv')

```

4.1 Clean Whitespaces

```
# Check whether there are whitespace value in the dataframe
df1.iloc[:, -1].values[0].isspace()
```

Output:

True

```
# Clean whitespaces
for col in range(df1.shape[1]):
    if df1.iloc[:, col].dtype == 'object':
        for index, val in enumerate(df1.iloc[:, col].values):
            df1.iloc[:, col].values[index] = str(val).strip()
            if str(val).isspace() or val is None or str(val).lower() == 'nan'
or str(val).lower() == 'none' or val == '' or str(val) == "/" or str(val) ==
"NaT":
                df1.iloc[:, col].values[index] = np.NaN
```

```
# Check whether the value is whitespace again
df1.iloc[:, -1].values[0].isspace()
```

Output:

False

```
# Save into a CSV File
df1.to_csv('2018-2021_dataset_new_cleaned_whitespace.csv')
```

4.2 Drop Duplicate POLICY_NO

```
df1 = pd.read_csv(r'2018-2021_dataset_new_cleaned_whitespace.csv', index_col=0)
df1
```

```
# Drop duplicate
print('Before:', df1[df1[['POLICY_NO']].duplicated()].shape[0])
df1 = df1.sort_values('EarlyClaimAndLapsed',
ascending=False).drop_duplicates(subset=['POLICY_NO'],
keep='first').sort_index()
print('After: ', df1[df1[['POLICY_NO']].duplicated()].shape[0])
```

Output:

Before: 866612

After: 0

```
print('Total rows: {}'.format(df1.shape[0]))
```

Output:

Total rows: 256661

```
df1['EarlyClaimAndLapsed'].value_counts()
```

Output:

N 26128

Y 1267

```
df1['EarlyClaimAndLapsed'].isnull().sum()
```

Output:

229266

```
values = {"EarlyClaimAndLapsed": 'N'}  
df1 = df1.fillna(value=values)
```

```
# Convert EarlyClaimAndLapsed from Y & N to 1 & 0  
df1['EarlyClaimAndLapsed'] = df1['EarlyClaimAndLapsed'].apply(lambda x: 1 if x  
== 'Y' else 0)
```

```
df1['EarlyClaimAndLapsed'].value_counts()
```

Output:

0 255394

1 1267

```
df1.reset_index(inplace=True, drop=True)  
df1.head()
```

```
df_full = pd.read_csv('2018-2021_dataset_new (whole_data).csv')
```

```
# Select these features based on the finding from the previous results &  
experiences.  
df1 = df1[['POLICY_NO', 'SEX', 'RSK_SUM_ASSURE', 'OCCUPATION_CLASS', 'RACE',
```

```
'MARITAL_STATUS', 'BMI',
      'POL HOLDER_TYPE', 'PAYMENT_MODE',
'PAYMENT_METHOD', 'SELL_AGENT_AGE', 'SELL_AGENT_BRANCH_CODE',
      'SELL_AGENT_RACE', 'SELL_AGENT_EDU_LEVEL', 'RISK_TERM', 'STATE',
'EarlyClaimAndLapsed']]

df1.info()
```

4.3 Fill in the Blank (RACE)

Example

BEFORE

ROW	POLICY_NO	RACE
1	1234	CH
2	1234	
3	1234	
4	1234	CH

AFTER

ROW	POLICY_NO	RACE
1	1234	CH
2	1234	CH
3	1234	CH
4	1234	CH

```
df1.isnull().sum()
```

Output:

POLICY_NO	0
SEX	0
RSK_SUM_ASSURE	0
OCCUPATION_CLASS	743
RACE	45929
MARITAL_STATUS	0
BMI	2872
POL HOLDER_TYPE	0
PAYMENT_MODE	0

```

PAYMENT_METHOD          0
SELL_AGENT_AGE          1532
SELL_AGENT_BRANCH_CODE  0
SELL_AGENT_RACE          2074
SELL_AGENT_EDU_LEVEL    25092
RISK_TERM                0
STATE                    244
EarlyClaimAndLapsed     0
dtype: int64

```

```

for i in df1['POLICY_NO'].loc[(df1['RACE'].isnull() == True)].unique():

    if len(df_full['RACE'].loc[(df_full['POLICY_NO'] == i)].value_counts())
    !=0:
        replace_val = df_full['RACE'].loc[(df_full['POLICY_NO'] ==
i)].value_counts().keys()[0]
        df1['RACE'].loc[(df1['POLICY_NO'] == i)] = replace_val

```

```
df1.isnull().sum()
```

Output:

```

POLICY_NO              0
SEX                    0
RSK_SUM_ASSURE         0
OCCUPATION_CLASS       17
RACE                   13
MARITAL_STATUS         0
BMI                    151
POL HOLDER_TYPE        0
PAYMENT_MODE           0
PAYMENT_METHOD         0
SELL_AGENT_AGE         125
SELL_AGENT_BRANCH_CODE  0
SELL_AGENT_RACE        133
SELL_AGENT_EDU_LEVEL   2767
SELL_AGENT_OCCUPATION_CODE 125
RISK_TERM              0
STATE                  7
EarlyClaimAndLapsed    0
dtype: int64

```

```
df1 = df1.dropna()
df1.shape
```

Output:

(24484 18)

4.4 Handling Outliers

```
num_col = ['RISK_TERM', 'BMI', 'SELL_AGENT_AGE']

#Outliers detection
#Show data in boxplots
Numeric_dic = {}
for x in num_col:
    Numeric_dic[x]= df1[x].to_numpy()

for x in Numeric_dic:
    fig = plt.figure()
    fig.suptitle(x, fontsize=14, fontweight='bold')
    ax = fig.add_subplot(111)
    ax.boxplot(Numeric_dic[x])
    plt.show()
```

```
#Remove outliers
for x in num_col:
    Q1 = df1[x].quantile(0.25)
    Q3 = df1[x].quantile(0.75)
    IQR = Q3 - Q1
    max_range = (Q3+ (1.5*IQR))
    min_range = (Q1- (1.5*IQR))
    df1[x] = df1[x].loc[(df1[x] < max_range)]
    df1[x] = df1[x].loc[(df1[x] > min_range)]

#remove missing values again
df1 = df1.dropna()
```

```
#Remove outliers
for x in num_col:
    Q1 = df1[x].quantile(0.25)
    Q3 = df1[x].quantile(0.75)
    IQR = Q3 - Q1
    max_range = (Q3+ (1.5*IQR))
    min_range = (Q1- (1.5*IQR))
    df1[x] = df1[x].loc[(df1[x] < max_range)]
    df1[x] = df1[x].loc[(df1[x] > min_range)]

#remove missing values again
df1 = df1.dropna()
```

```
#Show data in boxplots
Numeric_dic = {}
for x in num_col:
    Numeric_dic[x]= df1[x].to_numpy()

for x in Numeric_dic:
    fig = plt.figure()
    fig.suptitle(x, fontsize=14, fontweight='bold')
    ax = fig.add_subplot(111)
    ax.boxplot(Numeric_dic[x])
    plt.show()
```

```
df1['RACE'].value_counts()
```

Output:

```
IN    12666
CH     9248
MA     1182
OT      232
BN      160
      14
Name: RACE, dtype: int64
```

```
# Drop empty Race
df1 = df1.loc[(df1['RACE']!=" ")]
```

4.5 Feature Encoding

```
df1['RSK_SUM_ASSURE'] = df1['RSK_SUM_ASSURE'].astype(str)

rsa_arr = ['100', '150', '200', '300', '400', '600']
df1['RSK_SUM_ASSURE'] = df1['RSK_SUM_ASSURE'].apply(lambda x: -999 if x not in
rsa_arr else x)

df1['RSK_SUM_ASSURE'] = df1['RSK_SUM_ASSURE'].replace({'100': 1, '150': 1.5,
'200': 2, '300': 3, '400': 4,
'600': 6}).astype(np.float32)
```

```
## Customer State: Replace Foreign countries with Other
foreign_countries =
['SINGAPORE', 'BRUNEI', 'AUSTRALIA', 'U.S.A.', 'THAILAND', 'UNITED
KINGDOM', 'TAIWAN', 'NEW ZEALAND']
df1['STATE'] = df1['STATE'].apply(lambda x: 'OTHER' if x in foreign_countries
else x)
print(df1['STATE'].value_counts())
```

```
df1['BRANCH_STATE'] = df1['SELL_AGENT_BRANCH_CODE'].replace({'J1':
'Johor', 'P2': 'Penang', 'A1': 'Perak', 'N1': 'Negeri Sembilan',
'M1': 'Melacca', 'P1':
'Penang', 'W1': 'Kuala Lumpur', 'S2': 'Sarawak',
'S1': 'Sarawak', 'B1':
'Selangor', 'K1': 'Kedah', 'D1': 'Kelantan',
'C1': 'Kuantan', 'S3':
'Sarawak', 'H2': 'Sabah', 'H1': 'Sabah',
'KE': 'Financial
Advisor', 'FA': 'Financial Advisor', 'RH': 'RHB', 'HO': 'RHB'
}))
```

```

branch_state_list = ['Johor', 'Penang', 'Perak', 'Negeri Sembilan',
                    'Melacca', 'Penang', 'Kuala Lumpur', 'Sarawak',
                    'Sarawak', 'Selangor', 'Kedah', 'Kelantan',
                    'Kuantan', 'Sarawak', 'Sabah', 'Sabah', 'Financial
Advisor', 'RHB']

df1['BRANCH_STATE'] = df1['BRANCH_STATE'].apply(lambda x: 'RHB' if x not in
branch_state_list else x)

print(df1['BRANCH_STATE'].value_counts())

```

```

df1 = df1.drop(columns = ['SELL_AGENT_BRANCH_CODE'])

```

```

import json
def mapOccupation(df):
    # Opening JSON file
    f = open('occupationDict.json')

    # Returns JSON object as a dictionary
    occ_dict = json.load(f)

    for key, value_list in occ_dict.items():
        for i, row in df.iterrows():

            if row['SELL_AGENT_OCCUPATION_CODE'] in value_list:
                df.loc[i, 'SELL_AGENT_OCCUPATION_CODE'] = key

    df = df.dropna(subset=['SELL_AGENT_OCCUPATION_CODE'])
    df.reset_index(inplace=True, drop=True)

    options = ['A', 'B', 'C', 'D']
    for x in range(df.shape[0]):
        if df['SELL_AGENT_OCCUPATION_CODE'].iloc[x] not in options:
            df['SELL_AGENT_OCCUPATION_CODE'].iloc[x] = 'Undefined'

```

```
return df
```

```
df1 = mapOccupation(df1)  
df1
```

```
df1['SELL_AGENT_OCCUPATION_CODE'].value_counts()
```

Output:

```
B          17177  
Undefined   3908  
A           2202  
C           182  
D            19  
Name: SELL_AGENT_OCCUPATION_CODE, dtype: int64
```

```
df1['RACE'].value_counts()
```

Output:

```
IN    12666  
CH     9248  
MA     1182  
OT      232  
BN      160  
Name: RACE, dtype: int64
```

```
df1['RACE'] = df1['RACE'].replace({'BN': 'OT'})  
df1['SELL_AGENT_RACE'] = df1['SELL_AGENT_RACE'].replace({'BN': 'OT'})
```

```
df1['RACE'].value_counts()
```

Output:

```
IN    12666  
CH     9248  
MA     1182  
OT      392  
Name: RACE, dtype: int64
```

```
df1['SELL_AGENT_RACE'].value_counts()
```

Output:

```
IN    13024  
CH     9742
```

```
MA      628
OT      94
Name: SELL_AGENT_RACE, dtype: int64
```

```
df1.reset_index(inplace=True, drop=True)
df1
```

```
df1.to_csv('FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022).csv')
```

4.6 Train Test Split

```
df2 = pd.read_csv(r'FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022).csv', index_col=0)
df2
```

```
df2.reset_index(inplace=True, drop=True)
df2 = df2.drop(columns = ['POLICY_NO'])
```

```
df_test['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    22296
1     1192
Name: EarlyClaimAndLapsed, dtype: int64
```

```
X = df2.drop(columns = ['EarlyClaimAndLapsed'])
```

```
y = df2['EarlyClaimAndLapsed']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
stratify=y, shuffle=True, random_state = 44)
```

```
holder_type_freq_dict = X_train.groupby('POL_HOLDER_TYPE').size()/len(X_train)
holder_type_freq_dict
```

Output:

```
POL_HOLDER_TYPE
AG    0.980788
BC    0.011815
```

```
FA    0.001543
ST    0.005854
dtype: float64
```

```
paymode_freq_dict = X_train.groupby('PAYMENT_MODE').size()/len(X_train)
paymode_freq_dict
```

Output:

```
PAYMENT_MODE
1    0.125492
2    0.040820
4    0.034380
12   0.799308
dtype: float64
```

```
marital_freq_dict = X_train.groupby('MARITAL_STATUS').size()/len(X_train)
marital_freq_dict
```

Output:

```
MARITAL_STATUS
D    0.010963
M    0.473071
N    0.015753
S    0.444332
W    0.015008
X    0.040873
dtype: float64
```

```
paymethod_freq_dict = X_train.groupby('PAYMENT_METHOD').size()/len(X_train)
paymethod_freq_dict
```

Output:

```
PAYMENT_METHOD
B    0.005269
C    0.249548
D    0.080096
G    0.000053
R    0.665035
dtype: float64
```

```
edu_freq_dict = X_train.groupby('SELL_AGENT_EDU_LEVEL').size()/len(X_train)
edu_freq_dict
```

Output:

```
SELL_AGENT_EDU_LEVEL
H    0.000426
P    0.006759
S    0.579510
T    0.376583
U    0.036722
dtype: float64
```

```
state_freq_dict = X_train.groupby('STATE').size()/len(X_train)
state_freq_dict
```

Output:

```
STATE
JOHOR          0.189888
KEDAH          0.059074
KELANTAN       0.009899
KUALA LUMPUR   0.025386
LABUAN         0.000373
MELAKA         0.081107
NEGERI SEMBILAN 0.065407
OTHER          0.000692
PAHANG         0.020064
PENANG         0.164183
PERAK          0.138797
PERLIS         0.001011
PUTRAJAYA      0.000426
SABAH          0.004577
SARAWAK        0.104151
SELANGOR       0.134007
TERENGGANU     0.000958
dtype: float64
```

```
branch_state_freq_dict = X_train.groupby('BRANCH_STATE').size()/len(X_train)
branch_state_freq_dict
```

Output:

```
BRANCH_STATE
Financial Advisor  0.001490
Johor              0.170144
Kedah              0.027408
Kelantan           0.011336
Kuala Lumpur       0.082171
Kuantan            0.011123
Melacca            0.092123
```

```

Negeri Sembilan      0.099095
Penang                0.205322
Perak                 0.128579
RHB                   0.017669
Sabah                 0.003725
Sarawak               0.108196
Selangor              0.041618
dtype: float64

```

```

# Frequency encoding
freq_encode_cols = ['POL HOLDER TYPE', 'PAYMENT_MODE', 'PAYMENT_METHOD',
                    'MARITAL_STATUS',
                    'SELL_AGENT_EDU_LEVEL', 'STATE', 'BRANCH_STATE']
for i in freq_encode_cols:
    freq_dict = X_train.groupby(i).size()/len(X_train)
    X_train.loc[:, '{}'.format(i)] = X_train[i].map(freq_dict)
    X_test.loc[:, '{}'.format(i)] = X_test[i].map(freq_dict)

```

```

from sklearn.preprocessing import OneHotEncoder, MinMaxScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer

full_processor = ColumnTransformer(
    transformers=[
        ("num", MinMaxScaler(), ['RISK_TERM', 'BMI', 'SELL_AGENT_AGE',
        'RSK_SUM_ASSURE']),
        ("nom", OneHotEncoder(handle_unknown="ignore", sparse=False), ['SEX',
        'RACE', 'SELL_AGENT_RACE']),
        ("occ", OrdinalEncoder(categories=[['A', 'B', 'C', 'D']]),
        ['OCCUPATION_CLASS']),
        ("oc2", OrdinalEncoder(categories=[['A', 'B', 'C', 'D', 'Undefined']]),
        ['SELL_AGENT_OCCUPATION_CODE']),
    ], remainder= 'passthrough'
)

```

```

full_processor.fit(X_train)

```

```
X_train = full_processor.fit_transform(X_train)
X_train
```

```
X_test = full_processor.transform(X_test)
X_test
```

```
feature_names = full_processor.get_feature_names_out()
feature_names
```

```
import re
count_Remainder = 0
for i in range(feature_names.size):
    if re.match(r'^remainder', feature_names[i]):
        count_Remainder += 1
count_Remainder

get_name = {}
for i in range(feature_names.size - count_Remainder):
    get_name[i] = feature_names[i][5:]
```

```
get_name_append_list = ['MARITAL_STATUS', 'POL HOLDER_TYPE', 'PAYMENT_MODE',
                        'PAYMENT_METHOD',
                        'SELL_AGENT_EDU_LEVEL', 'STATE', 'BRANCH_STATE']
idx = len(get_name) - 1
for i in get_name_append_list:
    get_name[idx+1] = i
    idx += 1
get_name
```

```
feature_name_list=list(get_name.values())
print(feature_name_list)
```

```
X_train_df = pd.DataFrame(data= X_train, columns= feature_name_list)
X_train_df
```

```
X_test_df = pd.DataFrame(data= X_test, columns= feature_name_list)
X_test_df
```

```
y_train.reset_index(inplace=True, drop=True)
y_test.reset_index(inplace=True, drop=True)
```

```
# Concat X_test_df and y_test to a testing dataframe
test_df = pd.concat([X_test_df,y_test], axis=1)
test_df.reset_index(inplace=True, drop=True)
test_df
```

```
test_df['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    4460
1     238
Name: EarlyClaimAndLapsed, dtype: int64
```

```
# Concat X_train_df and y_train to be a training dataframe, and generate more
sample using Generative Adversarial Networks
train_df = pd.concat([X_train_df,y_train], axis=1)
train_df.reset_index(inplace=True, drop=True)
train_df
```

```
train_df['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    17836
1     954
Name: EarlyClaimAndLapsed, dtype: int64
```

```
train_df.to_csv('FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022) TrainDF.csv')
test_df.to_csv('FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022) TestDF.csv')
```

4.7 Prepare 2018-2021 Test Data

```
test_whole_data_df = pd.read_csv(r'FULL_NOTEBOOK_DATA_TO_TRAIN  
(14-11-2022).csv', index_col=0)  
test_whole_data_df
```

```
test_whole_data_df.reset_index(inplace=True, drop=True)  
test_whole_data_df_target = test_whole_data_df['EarlyClaimAndLapsed']  
test_whole_data_df_target
```

```
test_whole_data_df = test_whole_data_df.drop(columns = ['POLICY_NO',  
'EarlyClaimAndLapsed'])  
test_whole_data_df
```

```
# Frequency Encoding  
test_whole_data_df.loc[:, '{}'.format('MARITAL_STATUS')] =  
test_whole_data_df['MARITAL_STATUS'].map(marital_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('POL HOLDER_TYPE')] =  
test_whole_data_df['POL HOLDER_TYPE'].map(holder_type_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('PAYMENT_MODE')] =  
test_whole_data_df['PAYMENT_MODE'].map(paymode_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('PAYMENT_METHOD')] =  
test_whole_data_df['PAYMENT_METHOD'].map(paymethod_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('SELL_AGENT_EDU_LEVEL')] =  
test_whole_data_df['SELL_AGENT_EDU_LEVEL'].map(edu_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('STATE')] =  
test_whole_data_df['STATE'].map(state_freq_dict)  
test_whole_data_df.loc[:, '{}'.format('BRANCH_STATE')] =  
test_whole_data_df['BRANCH_STATE'].map(branch_state_freq_dict)
```

```
test_whole_data_df = full_processor.transform(test_whole_data_df)  
test_whole_data_df
```

```
test_whole_data_df2 = pd.DataFrame(data= test_whole_data_df, columns=  
feature_name_list)
```

```
test_whole_data_df2.reset_index(inplace=True, drop=True)
test_whole_data_df2
```

```
test_whole_data_df2 =
pd.concat([test_whole_data_df2, test_whole_data_df_target], axis=1)
test_whole_data_df2.reset_index(inplace=True, drop=True)
test_whole_data_df2
```

```
test_whole_data_df2['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    22296
1     1192
Name: EarlyClaimAndLapsed, dtype: int64
```

```
test_whole_data_df2.to_csv('FULL_DATA_TO_TEST(14-11-2022).csv')
```

4.8 GAN

```
train_df = pd.read_csv(r'FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022) TrainDF.csv',
index_col=0)
test_df = pd.read_csv(r'FULL_NOTEBOOK_DATA_TO_TRAIN (14-11-2022) TestDF.csv',
index_col=0)
```

```
train_df = train_df.drop(columns = ['SEX_F', 'SEX_M', 'SEX_U',
'OCCUPATION_CLASS', 'BRANCH_STATE', 'SELL_AGENT_RACE_MA',
'RACE_MA', 'SELL_AGENT_RACE_OT', 'RACE_OT', 'SELL_AGENT_OCCUPATION_CODE'])
test_df = test_df.drop(columns = ['SEX_F', 'SEX_M', 'SEX_U', 'OCCUPATION_CLASS',
'BRANCH_STATE', 'SELL_AGENT_RACE_MA', 'RACE_MA', 'SELL_AGENT_RACE_OT', 'RACE_OT',
'SELL_AGENT_OCCUPATION_CODE'])
```

```
majority_cls = train_df.loc[train_df['EarlyClaimAndLapsed']==0]
majority_cls
```

```
minority_cls = train_df.loc[train_df['EarlyClaimAndLapsed']==1]
minority_cls
```

```
cat_columns = {'RACE_CH', 'RACE_IN', 'SELL_AGENT_RACE_CH',
               'SELL_AGENT_RACE_IN'}

# Split into training and test sets
df_x_train, df_x_test, df_y_train, df_y_test = train_test_split(
    minority_cls.drop("EarlyClaimAndLapsed", axis=1),
    minority_cls["EarlyClaimAndLapsed"],
    test_size=0.20,
    #shuffle=False,
    random_state=42,
)

# Create dataframe versions for tabular GAN
df_x_test, df_y_test = df_x_test.reset_index(drop=True),
df_y_test.reset_index(drop=True)
df_y_train = pd.DataFrame(df_y_train)
df_y_test = pd.DataFrame(df_y_test)

gen_x, gen_y = GANGenerator(gen_x_times=5, cat_cols=cat_columns,
    bot_filter_quantile=0.001, top_filter_quantile=0.999, \
    is_post_process=True,
    adversarial_model_params={
        "metrics": "AUC", "max_depth": 12, "max_bin": 100,
        "learning_rate": 0.02, "random_state": \
        49, "n_estimators": 800,
    }, pregeneration_frac=2, only_generated_data=False, \
    gan_params = {"batch_size": 500, "patience": 30, \
    "epochs" : 500, "objective":
"binary"}).generate_data_pipe(df_x_train, df_y_train, \
    df_x_test, deep_copy=True, only_adversarial=False, \
    use_adversarial=True)
```

```
gen_y.value_counts()
```

Output:

```
0    1146
1     906
Name: EarlyClaimAndLapsed, dtype: int64
```

```
gen_y = gen_y.replace(0,1)
gen_y
```

```
gen_y.value_counts()
```

Output:

```
1    2052
Name: EarlyClaimAndLapsed, dtype: int64
```

```
merged_gen_XY = pd.concat([gen_x,gen_y], axis = 1)
merged_gen_XY
```

```
merged_gen_XY['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
1    2052
Name: EarlyClaimAndLapsed, dtype: int64
```

```
majority_cls.reset_index(inplace=True, drop=True)
majority_cls
```

```
merged_XY = pd.concat([merged_gen_XY, majority_cls])
merged_XY
```

```
merged_XY.reset_index(inplace=True, drop=True)
merged_XY
```

```
merged_XY['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    17836
1     2052
Name: EarlyClaimAndLapsed, dtype: int64
```

4.9 Random Sampling

```
def random_sampling(train_df, test_df, cat_columns):
    arr_modelling = {}

    #Create a dictionary to map the column name with 'category'
    value='category'
    cat_col_dict = dict.fromkeys(cat_columns, value)

    for j in range(0,499):
        # =====TRAINING DATASET=====
        df_1 = train_df.loc[(train_df['EarlyClaimAndLapsed'] ==
0)].sample(n=train_df['EarlyClaimAndLapsed'].value_counts()[1]+1000,
random_state =j)
        df_2 = train_df.loc[(train_df['EarlyClaimAndLapsed'] ==
1)].sample(n=train_df['EarlyClaimAndLapsed'].value_counts()[1], random_state
=j)
        df_con = pd.concat([df_1, df_2], axis=0)

        # Seperate the train data into X_train and y_train
        X_train = df_con.drop(columns = ['EarlyClaimAndLapsed'])
        y_train = df_con['EarlyClaimAndLapsed']

        # Change the categorical feature dtype to category
        X_train = X_train.astype(cat_col_dict)
        # =====END TRAINING DATASET=====

        # =====TESTING DATASET=====
        df_1b = test_df.loc[(test_df['EarlyClaimAndLapsed'] ==
0)].sample(n=test_df['EarlyClaimAndLapsed'].value_counts()[1]+500, random_state
=j)
        df_2b = test_df.loc[(test_df['EarlyClaimAndLapsed'] ==
1)].sample(n=test_df['EarlyClaimAndLapsed'].value_counts()[1], random_state =j)
        df_conb= pd.concat([df_1b, df_2b], axis=0)

        # Seperate the test data into X_test and y_test
        X_test = df_conb.drop(columns = ['EarlyClaimAndLapsed'])
```

```

y_test = df_conb['EarlyClaimAndLapsed']

# Change the categorical feature dtype to category
X_test = X_test.astype(cat_col_dict)
# =====END TESTING DATASET=====

params = {
    'n_estimators': 800,
    'max_delta_step': 14,
    'max_depth': 15,
    'learning_rate': 0.0853391091008654,
    'colsample_bytree': 0.7071749068404311,
    'subsample': 0.70673370189980923,
    'alpha': 3.7284037159008734,
    'reg_lambda': 0.001210408227269722,
    'gamma': 1.1598780502748447,
    'min_child_weight': 0.49033802075732813,
    'max_bin': 255,
    'tree_method': 'hist'
}

xgb_clf = XGBClassifier(callbacks=None, enable_categorical=True,
importance_type='gain', max_leaves=0,
                        missing=-999, n_jobs=-1, predictor='cpu_predictor',
                        random_state=99, scale_pos_weight=1, **params)
xgb_clf.fit(X_train,y_train, verbose=False)
y_test_pred = xgb_clf.predict(X_test)

score = precision_score(y_test, y_test_pred)
score1 = recall_score(y_test, y_test_pred)
score2 = accuracy_score(y_test, y_test_pred)
arr_modelling[j] = [score, score1, score2]
return arr_modelling

```

```

arr_modelling_df = pd.DataFrame.from_dict(random_sampling(merged_XY, test_df,
cat_columns))
arr_modelling_df

```

```
df1_transposed = arr_modelling_df.T
df1_transposed = df1_transposed.rename(columns={0: 'Precision', 1: 'Recall', 2:
'Accuracy'})
sorted_transposed = df1_transposed.sort_values(by=['Accuracy'],ascending=False)
sorted_transposed
```

```
RANDOM_STATE_VALUE = 302
## Train
df_1_train = merged_XY.loc[(merged_XY['EarlyClaimAndLapsed'] ==
0)].sample(n=merged_XY['EarlyClaimAndLapsed'].value_counts()[1]+1000,
random_state =RANDOM_STATE_VALUE)
df_2_train = merged_XY.loc[(merged_XY['EarlyClaimAndLapsed'] ==
1)].sample(n=merged_XY['EarlyClaimAndLapsed'].value_counts()[1], random_state
=RANDOM_STATE_VALUE)
df_con_train = pd.concat([df_1_train, df_2_train], axis=0)

## Test
# df2.reset_index(inplace=True, drop=True)
df_1_test = test_df.loc[(test_df['EarlyClaimAndLapsed'] ==
0)].sample(n=test_df['EarlyClaimAndLapsed'].value_counts()[1]+500, random_state
=RANDOM_STATE_VALUE)
df_2_test = test_df.loc[(test_df['EarlyClaimAndLapsed'] ==
1)].sample(n=test_df['EarlyClaimAndLapsed'].value_counts()[1], random_state
=RANDOM_STATE_VALUE)
df_con_test = pd.concat([df_1_test, df_2_test], axis=0)
```

```
df_con_train['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    17836
```

```
df_con_test['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0    17836
```

```
X_train = df_con_train.drop(columns = ['EarlyClaimAndLapsed'])
y_train = df_con_train['EarlyClaimAndLapsed']
```

```
X_test = df_con_test.drop(columns = ['EarlyClaimAndLapsed'])
y_test = df_con_test['EarlyClaimAndLapsed']
```

```
X_train.shape
```

Output:

```
(5104, 14)
```

```
X_test.shape
```

Output:

```
(976, 14)
```

```
y_train.value_counts()
```

Output:

```
0    3052
```

```
1     2052
```

```
Name: EarlyClaimAndLapsed, dtype: int64
```

```
y_test.value_counts()
```

Output:

```
0     738
```

```
1     238
```

```
Name: EarlyClaimAndLapsed, dtype: int64
```

5. Modeling

5.1 Handle Imbalanced Dataset

Although XGBoost can handle imbalance dataset, but it's always recommended to balance the dataset to prevent biases to the majority class. Thus, the parameter `scale_pos_weight` will be used to train a class-weighted or cost-sensitive version of XGBoost for imbalanced classification.

```
scale_pos_weight=(y_train.value_counts()[0])/(y_train.value_counts()[1])
```

Output:

1.4873294346978558

5.2 Hyperparameter Tuning

The purposes of hyperparameter tuning are:

1. Achieve high precision especially the minority class.
2. Prevent overfitting.

In this case, we utilize the Optuna which is an automatic hyperparameter optimization software framework, particularly designed for machine learning.

```
def cv_tuner(X, y):  
    """  
    :param X: array of training features  
    :param y: array of target label  
    :return: None  
    """  
  
    def train_evaluate(X, y, params):  
  
        xgb_model = XGBClassifier(**params)  
  
        # define evaluation  
        cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=1)  
  
        # evaluate model  
        scores = cross_val_score(xgb_model, X, y, cv=cv,  
scoring='balanced_accuracy', n_jobs=-1)  
        best_score = np.mean(scores)  
  
        return best_score  
  
    def objective(trial):  
  
        params = {  
            "verbosity": 1, # 0 (silent) - 3 (debug)  
            "objective": "binary:logistic",  
            "n_estimators": trial.suggest_int("n_estimators", 300,  
3000,step=100, log=False),
```

```

        "max_delta_step": trial.suggest_int("max_delta_step", 1, 15, step=1),
        "max_depth": trial.suggest_int("max_depth", 6, 24),
        "learning_rate": trial.suggest_loguniform("learning_rate", 0.01,
0.5),
        "colsample_bytree": trial.suggest_loguniform("colsample_bytree",
0.4, 0.6),
        "subsample": trial.suggest_loguniform("subsample", 0.4, 0.6),
        "alpha": trial.suggest_loguniform("alpha", 0.0001, 10.0),
        "lambda": trial.suggest_loguniform("lambda", 0.0001, 10.0),
        "gamma": trial.suggest_loguniform("gamma", 0.1, 10.0),
        "min_child_weight": trial.suggest_loguniform("min_child_weight",
0.1, 10),
        'max_bin': trial.suggest_categorical("max_bin", [63, 150, 255]),
        "seed": 42,
        "n_jobs": -1,
        "scale_pos_weight": scale_pos_weight,
        "eval_metric": ['aucpr', 'map', 'error'],
        "importance_type": 'gain',
        "missing": -999,
        'predictor': 'cpu_predictor',
        'enable_categorical': True,
        'tree_method': 'hist'
    }

    all_params = {**params}
    return train_evaluate(X, y, all_params)

# create the study object to set the direction of maximisation
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=300)

# Run statistics
print("Number of finished trials: ", len(study.trials))
print("Best trial:")
trial = study.best_trial

print("  Value: {}".format(trial.value))
print("  Params: ")

```

```

for key, value in trial.params.items():
    print("    {}: {}".format(key, value))

trial_df = study.trials_dataframe()
#print(trial_df)
print("\n Parameter
Importances", optuna.importance.get_param_importances(study))
fig = optuna.visualization.plot_param_importances(study)
fig.show()

```

```

# Run the Tuner
cv_tuner(X_train,y_train)

```

5.3 Model Evaluation

5.3.1 Tuned Parameters

```

params = {
    'n_estimators': 400,
    'max_delta_step': 14,
    'max_depth': 15,
    'learning_rate': 0.0853391091008654,
    'colsample_bytree': 0.4071749068404311,
    'subsample': 0.40673370189980923,
    'alpha': 3.7284037159008734,
    'reg_lambda': 0.001210408227269722,
    'gamma': 1.1598780502748447,
    'min_child_weight': 0.49033802075732813,
    'max_bin': 255,
    'tree_method': 'hist'
}

```

5.3.2 Result for RESAMPLED Data

```

cat_columns = {'SEX_M', 'SEX_U', 'RACE_CH', 'RACE_IN', 'RACE_MA',

```

```

        'SELL_AGENT_RACE_CH', 'SELL_AGENT_RACE_IN',
        'SELL_AGENT_RACE_MA', 'OCCUPATION_CLASS'}

value='category'
cat_col_dict = dict.fromkeys(cat_columns, value)
X_train = X_train.astype(cat_col_dict)
X_test = X_test.astype(cat_col_dict)

eval_set = [(X_train, y_train), (X_test,y_test)]
xgb_clf = XGBClassifier(callbacks=None, enable_categorical=True,
importance_type='gain', max_leaves=0,
                        missing=-999, n_jobs=-1, predictor='cpu_predictor',
                        random_state=99, scale_pos_weight=scale_pos_weight,
**params)
xgb_clf.fit(X_train,y_train, eval_metric=['error','logloss'],
eval_set=eval_set, early_stopping_rounds=20, verbose=False)
y_test_pred = xgb_clf.predict(X_test)
print(classification_report(y_test,y_test_pred))

#Generate the confusion matrix
cf_matrix = confusion_matrix(y_test, y_test_pred)

ax = sns.heatmap(cf_matrix, annot=True, cmap='Blues', fmt='g')

ax.set_title('Seaborn Confusion Matrix with labels\n\n');
ax.set_xlabel('\nPredicted Values')
ax.set_ylabel('Actual Values ');

## Ticket labels - List must be in alphabetical order
ax.xaxis.set_ticklabels(['False','True'])
ax.yaxis.set_ticklabels(['False','True'])

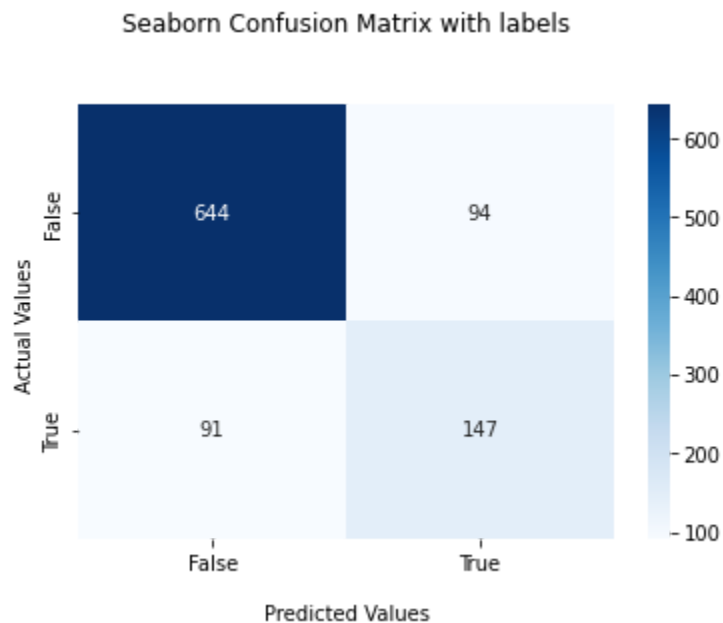
## Display the visualization of the Confusion Matrix.
plt.show()

```

Output:

	precision	recall	f1-score	support
0	0.88	0.87	0.87	738

	1	0.61	0.62	0.61	238
accuracy				0.81	976
macro avg		0.74	0.75	0.74	976
weighted avg		0.81	0.81	0.81	976



```

results = xgb_clf.evals_result()
# plot learning curves
plt.plot(results['validation_0']['logloss'], label='train_logloss')
plt.plot(results['validation_1']['logloss'], label='test_logloss')
plt.ylabel("Log Loss")
plt.title("XGBoost Log Loss")
plt.ylim([.1, .7])
# show the legend
plt.legend()
# show the plot
plt.show()

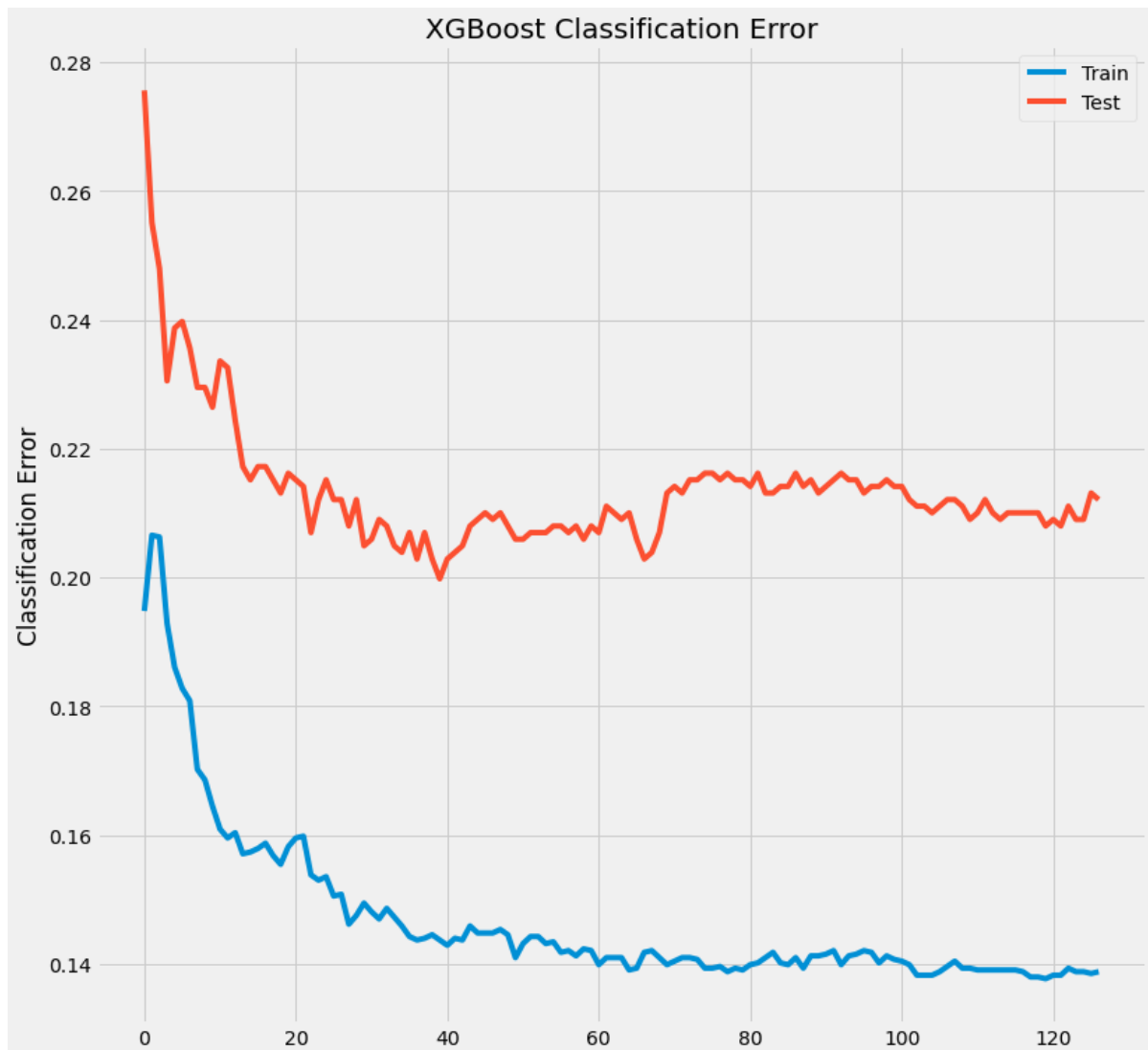
```

Output:



```
# plot classification error
epochs = len(results["validation_0"]["error"])
x_axis = range(0, epochs)
fig, ax = plt.subplots(figsize=(12,12))
ax.plot(x_axis, results["validation_0"]["error"], label="Train")
ax.plot(x_axis, results["validation_1"]["error"], label="Test")
ax.legend()
plt.ylabel("Classification Error")
plt.title("XGBoost Classification Error")
plt.figure(figsize=(12, 12), dpi=1000)
plt.show()
```

Output:



```
y_train_pred = xgb_clf.predict(X_train)
y_test_pred = xgb_clf.predict(X_test)

print("Accuracy Train: {:.4f}\nAccuracy Test:
{:.4f}".format(accuracy_score(y_train, y_train_pred),
                                                         accuracy_score(y_test,
y_test_pred)))
print()
print("Precision Train: {:.4f}\nPrecision Test:
{:.4f}".format(precision_score(y_train, y_train_pred),
                                                         precision_score(y_test,
y_test_pred)))
```

```

print()
print("Recall Train: {:.4f}\nRecall Test: {:.4f}".format(recall_score(y_train,
y_train_pred),
                                                    recall_score(y_test,
y_test_pred)))
print()
print("F1 Train: {:.4f}\nF1 Test: {:.4f}".format(f1_score(y_train,
y_train_pred),
                                                    f1_score(y_test,
y_test_pred)))
print()
print("AUC Train: {:.4f}\nAUC Test: {:.4f}".format(roc_auc_score(y_train,
xgb_clf.predict_proba(X_train)[: ,1]),
                                                    roc_auc_score(y_test,
xgb_clf.predict_proba(X_test)[: ,1])))

```

Output:

Accuracy Train: 0.8668

Accuracy Test: 0.8105

Precision Train: 0.7875

Precision Test: 0.6100

Recall Train: 0.8422

Recall Test: 0.6176

F1 Train: 0.8140

F1 Test: 0.6138

AUC Train: 0.9481

AUC Test: 0.8411

```

def Calculate_Plot_ROC(y_test, y_pred, name):
    # Calculate ROC curve
    fpr, tpr, thr = roc_curve(y_test, y_pred)
    roc_auc = auc(fpr,tpr)

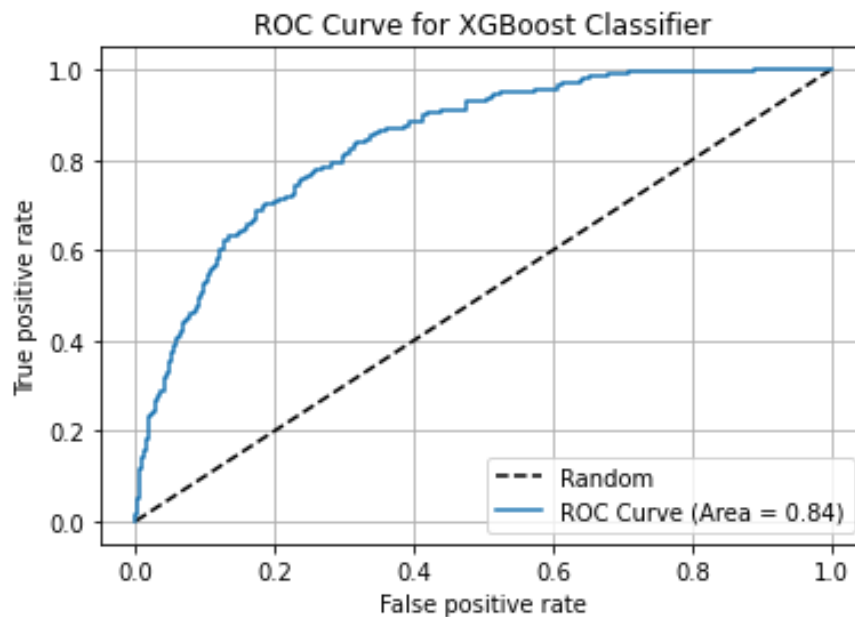
    # Plot the ROC curve
    plt.plot([0, 1], [0, 1], 'k--', label = 'Random')
    plt.plot(fpr, tpr, label ='ROC Curve (Area = %0.2f)' %roc_auc)

```

```
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title(f'ROC Curve for {name}')
plt.grid()
plt.legend(loc = 'best')
```

```
Calculate_Plot_ROC(y_test, xgb_clf.predict_proba(X_test)[: , 1], 'XGBoost Classifier')
```

Output:



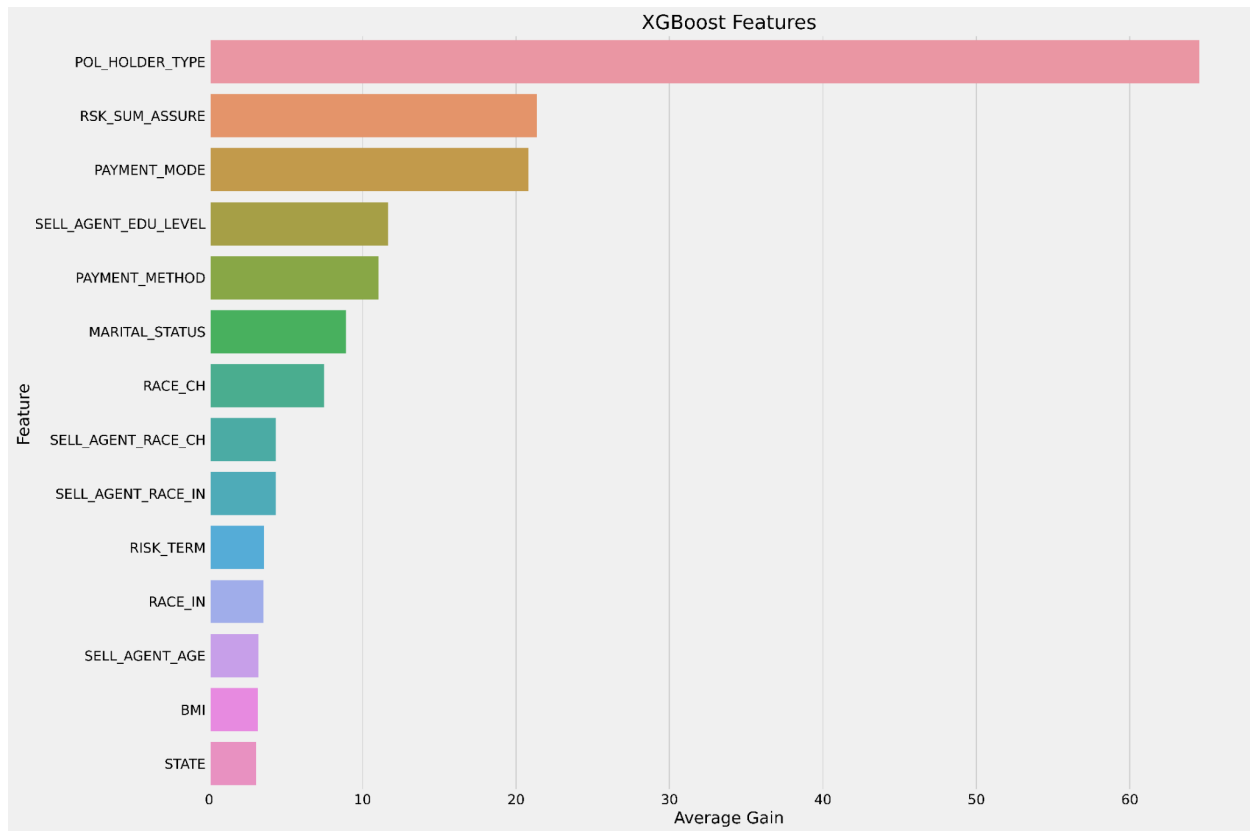
5.3.3 Feature Evaluation

```
feature_imp =
pd.DataFrame(sorted(zip(np.array(list(xgb_clf._Booster.get_score(importance_type='gain').values())),X_train.columns)), columns=['Average Gain', 'Feature'])
#gain = average gain for each split among all the trees
feature_imp = feature_imp.sort_values(by=['Average Gain'], ascending=False)

plt.figure(figsize=(18, 12), dpi=1000)
sns.barplot(x="Average Gain", y="Feature",
data=feature_imp.sort_values(by="Average Gain", ascending=False))
plt.title('XGBoost Features')
```

```
plt.tight_layout()
plt.show()
```

Output:



#Feature selection

```
xgb_clf_rfecv = RFECV(xgb_clf, step=1, cv=StratifiedKFold(5), scoring  
='balanced_accuracy')
```

```
xgb_clf_rfecv.fit(X_train,y_train)
```

```
rfecv_score = []
```

```
for i in xgb_clf_rfecv.grid_scores_:  
    rfecv_score.append(i)
```

```
plt.style.use('fivethirtyeight')  
plt.figure(figsize=(12,6), dpi=100)
```

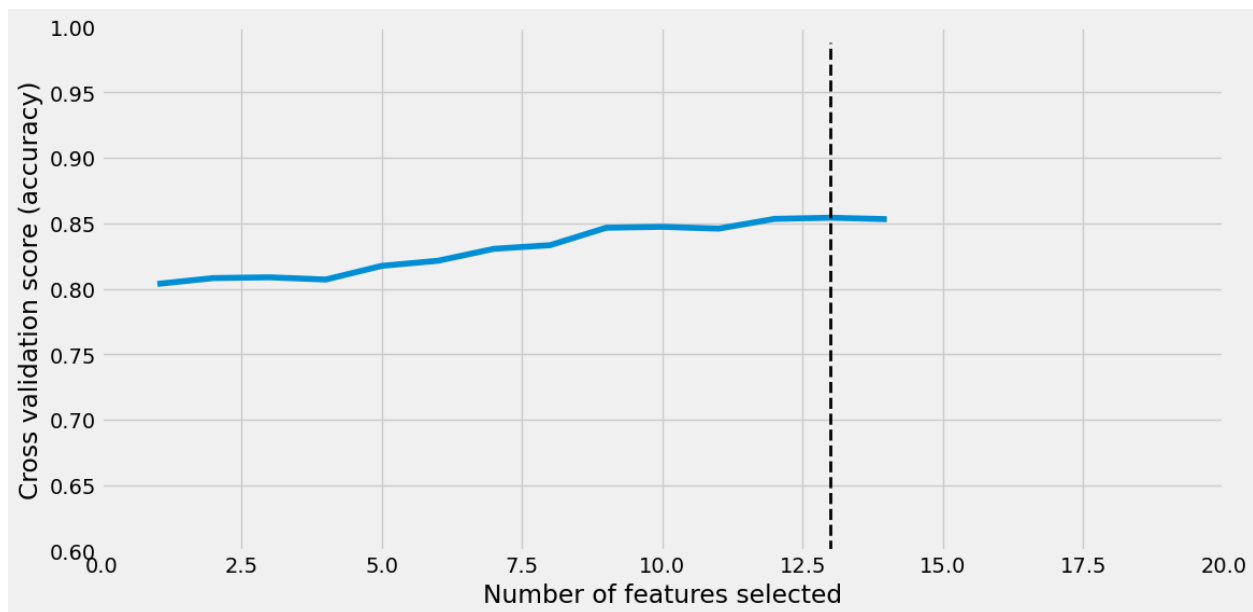
```

print("Optimal number of features : %d" % xgb_clf_rfecv.n_features_)
min_features_to_select = 1
# Plot number of features VS. cross-validation scores
plt.xlabel("Number of features selected")
plt.ylabel("Cross validation score (accuracy)")
plt.plot(
    range(min_features_to_select, len(rfecv_score) + min_features_to_select),
    rfecv_score)
plt.axvline(x=xgb_clf_rfecv.n_features_, ymin=0, ymax=0.97, color='black',
linestyle='dashed', linewidth = 2)
plt.xlim(0, 20)
plt.ylim(0.6,1.0)
plt.show()

```

Output:

Optimal number of features : 13



```

print('Selected features:')
print(xgb_clf_rfecv.support_)
print(xgb_clf_rfecv.ranking_)

```

Output:

Selected features:

```

[ True  True  True  True  True False  True  True  True  True  True  True
  True  True  True  True  True  True  True  True]
[1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1]

```

```
from BorutaShap import BorutaShap
selector = BorutaShap(importance_measure = 'shap', classification = True)
```

Output:

```
0      738
```

```
# Fits the selector
selector.fit(X = X_train, y = y_train, n_trials = 100, sample = True, verbose =
True)
# n_trials -> number of iterations for Boruta algorithm
# sample -> samples the data so it goes faster
```

Output:

```
13 attributes confirmed important: ['SELL_AGENT_EDU_LEVEL', 'RSK_SUM_ASSURE',
'PAYMENT_METHOD', 'PAYMENT_MODE', 'STATE', 'POL HOLDER_TYPE', 'RACE_CH',
'RACE_IN', 'SELL_AGENT_RACE_CH', 'SELL_AGENT_RACE_IN', 'SELL_AGENT_AGE',
'RISK_TERM', 'MARITAL_STATUS']
1 attributes confirmed unimportant: ['BMI']
0 tentative attributes remains: []
```

```
# Display features to be removed
features_to_remove = selector.features_to_remove
print(features_to_remove)
# Removes them
X_train_boruta_shap = X_train.drop(columns = features_to_remove)
X_test_boruta_shap = X_test.drop(columns = features_to_remove)
```

Output:

```
['SEX_M' 'SEX_U' 'RACE_MA' 'SELL_AGENT_RACE_MA' 'OCCUPATION_CLASS'
'BRANCH_STATE']
```

6. Test on 2018-2021 Data

```
test_df = pd.read_csv(r'FULL_DATA_TO_TEST(14-11-2022).csv', index_col=0)
test_df
```

```
test_df['EarlyClaimAndLapsed'].value_counts()
```

Output:

```
0      22296
1       1192
```

Name: EarlyClaimAndLapsed, dtype: int64

```
test_df = test_df.drop(columns = ['SEX_F', 'SEX_M', 'SEX_U', 'OCCUPATION_CLASS',  
'BRANCH_STATE', 'SELL_AGENT_RACE_MA', 'RACE_MA', 'SELL_AGENT_RACE_OT', 'RACE_OT',  
'SELL_AGENT_OCCUPATION_CODE'])
```

```
cat_columns = { 'RACE_CH', 'RACE_IN',  
                'SELL_AGENT_RACE_CH', 'SELL_AGENT_RACE_IN' }
```

#Independent variables

```
ori_X = test_df.drop(columns = ['EarlyClaimAndLapsed'])
```

#Create a dictionary to map the column name with 'category'

value='category'

```
cat_col_dict = dict.fromkeys(cat_columns, value)
```

```
ori_X = ori_X.astype(cat_col_dict)
```

#Dependent variable

```
ori_y = test_df['EarlyClaimAndLapsed']
```

Set Thresholder

```
pipe = make_pipeline(Thresholder(xgb_clf, 0.6))
```

```
pipe.fit(X_train, y_train)
```

#0.6

```
print('TEST ON ALL DATA')
```

```
y_train_pred = pipe.predict(ori_X)
```

```
print(classification_report(ori_y, y_train_pred))
```

#Generate the confusion matrix

```
cf_matrix = confusion_matrix(ori_y, y_train_pred)
```

```
ax = sns.heatmap(cf_matrix, annot=True, cmap='Blues', fmt='g')
```

```
ax.set_title('Seaborn Confusion Matrix with labels\n\n');
```

```
ax.set_xlabel('\nPredicted Values')
```

```
ax.set_ylabel('Actual Values ');
```

```

## Ticket labels - List must be in alphabetical order
ax.xaxis.set_ticklabels(['False','True'])
ax.yaxis.set_ticklabels(['False','True'])

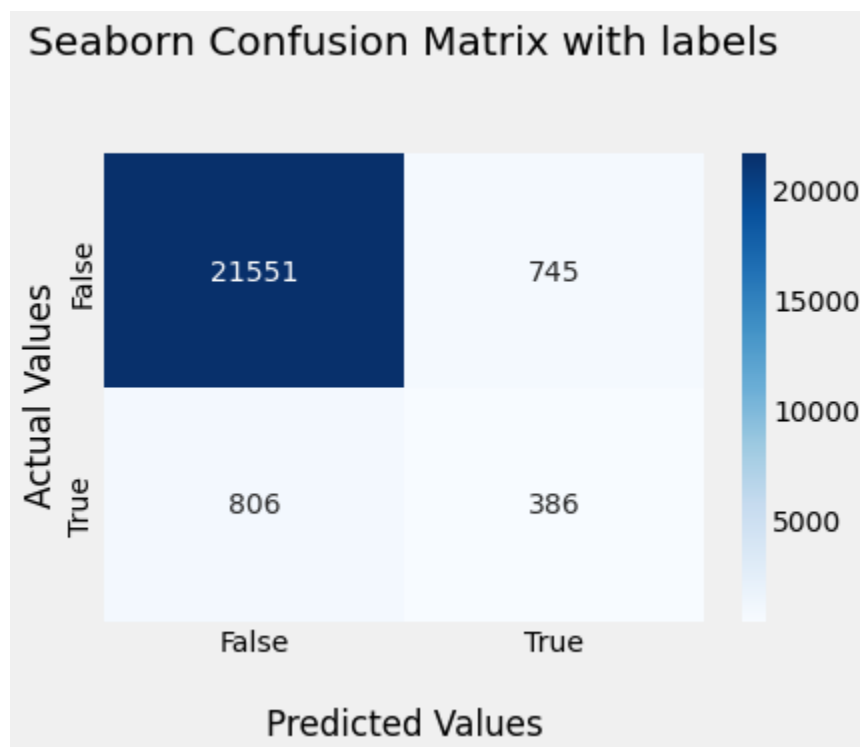
## Display the visualization of the Confusion Matrix.
plt.show()

```

Output:

TEST ON ALL DATA

	precision	recall	f1-score	support
0	0.96	0.97	0.97	22296
1	0.34	0.32	0.33	1192
accuracy			0.93	23488
macro avg	0.65	0.65	0.65	23488
weighted avg	0.93	0.93	0.93	23488

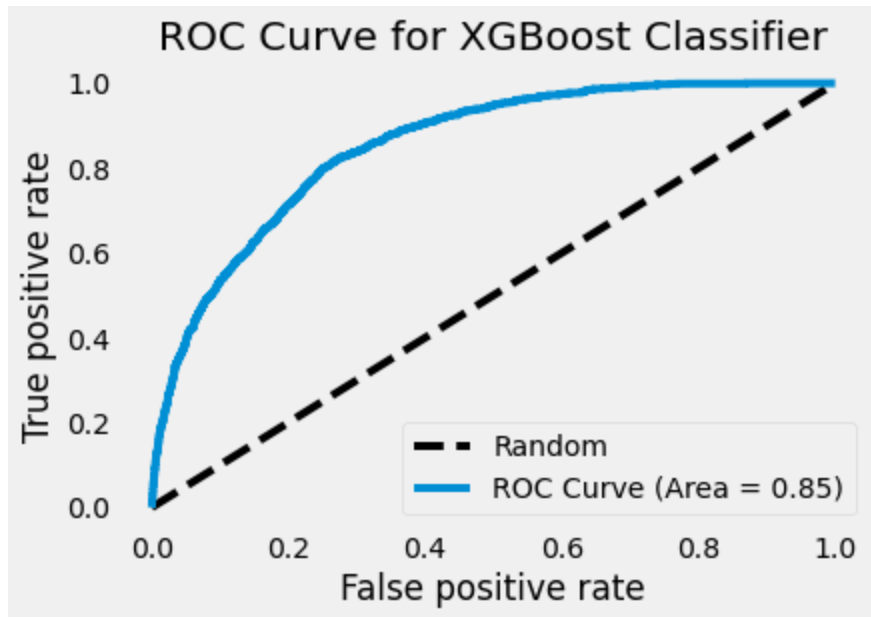


```

Calculate_Plot_ROC(ori_y, pipe.predict_proba(ori_X)[: , 1], 'XGBoost
Classifier')

```

Output:



```

y_train_pred = pipe.predict(X_train)
y_test_pred = pipe.predict(ori_X)

print("Accuracy Train: {:.4f}\nAccuracy Test:
{:.4f}".format(accuracy_score(y_train, y_train_pred),
                                accuracy_score(ori_y,
y_test_pred)))
print()
print("Precision Train: {:.4f}\nPrecision Test:
{:.4f}".format(precision_score(y_train, y_train_pred),
                                precision_score(ori_y,
y_test_pred)))
print()
print("Recall Train: {:.4f}\nRecall Test: {:.4f}".format(recall_score(y_train,
y_train_pred),
                                recall_score(ori_y,
y_test_pred)))
print()
print("F1 Train: {:.4f}\nF1 Test: {:.4f}".format(f1_score(y_train,
y_train_pred),
                                f1_score(ori_y,
y_test_pred)))
print()

```

```
print("AUC Train: {:.4f}\nAUC Test: {:.4f}".format(roc_auc_score(y_train,
pipe.predict_proba(X_train)[: ,1]),
                                                    roc_auc_score(ori_y,
pipe.predict_proba(ori_X)[: ,1])))
```

Output:

Accuracy Train: 0.9052
Accuracy Test: 0.9340

Precision Train: 0.9645
Precision Test: 0.3413

Recall Train: 0.7934
Recall Test: 0.3238

F1 Train: 0.8706
F1 Test: 0.3323

AUC Train: 0.9658
AUC Test: 0.8519

7. Save and Test Model

```
#Save the model as a pickle in a file
joblib.dump(pipe, 'XGB_ECTL_60TRH(17-11-2022).pkl')
```

```
load_clf = joblib.load(open(r'XGB_ECTL_60TRH(17-11-2022).pkl'))
```

```
data ={
    'RISK_TERM':0.235955,
    'BMI':0.513514,
    'SELL_AGENT_AGE':0.33871,
    'RSK_SUM_ASSURE':0.2,
    'RACE_CH': 1,
    'RACE_IN':0,
    'SELL_AGENT_RACE_CH':0,
    'SELL_AGENT_RACE_IN':1,
    'MARITAL_STATUS':0.444332,
    'POL HOLDER_TYPE':0.980788,
```

```

        'PAYMENT_MODE':0.799308,
        'PAYMENT_METHOD':0.665035,
        'SELL_AGENT_EDU_LEVEL':0.006759,
        'STATE':0.189888
    }
df = pd.DataFrame(data, index=[0])
def reindexDataFrame(df):
    columns_name = ['RISK_TERM', 'BMI', 'SELL_AGENT_AGE', 'RSK_SUM_ASSURE',
                    'RACE_CH', 'RACE_IN',
                    'SELL_AGENT_RACE_CH', 'SELL_AGENT_RACE_IN',
                    'MARITAL_STATUS', 'POL HOLDER_TYPE',
                    'PAYMENT_MODE', 'PAYMENT_METHOD', 'SELL_AGENT_EDU_LEVEL',
                    'STATE']
    return df.reindex(columns=columns_name)
df = reindexDataFrame(df)
df

```

```

prediction = pipe.predict(df)
prediction_proba = pipe.predict_proba(df)
print(prediction)
print(prediction_proba)

```

Output:

```

[1]
[[0.12363744 0.87636256]]

```

8. Deployment

8.1 Streamlit

Streamlit is a free and open-source Python library that enables us to deploy the Early Claim and Lapsed Prediction App with user-friendly graphical user interface in the fastest way. It doesn't require web development skills and it's compatible with the majority of Python libraries such as NumPy, Pandas and Scikit-Learn. Streamlit is available on <https://pypi.org/project/streamlit/> and we can download it via the CLI command “pip install streamlit”. Streamlit provides a lot of functionalities like write, dataframe, sidebar and input widgets. We utilized the power of Streamlit to build an interactive web application which allows users to make predictions easily. The user can upload the excel or csv file and the content and prediction will be displayed automatically. On the other hand, the users can make a single prediction by entering the

information manually at the sidebar. Moreover, the users can download the prediction results in excel format. To deliver a web application with a good user experience, we speed up the computation process by using a feature called `experimental_memo` to cache or memorize the model stored in the pickled file.

8.1.1 Installation Steps & Requirements

To run the web application, you should follow the following steps:

1. Download the Python version 3.8.10.
2. [OPTIONAL] Create a virtual environment with command “`virtualenv -p="C:\Python38\python.exe" venv`”.
3. Install Python packages with command “`pip install -r requirements.txt`”.
4. Run the web application with command “`streamlit run main.py`”.

** Note: Dependencies error, module not found error or version incompatible error may occur. Recommended to use Docker in section 5.2 as all the steps here can be done automatically.

8.1.2 App.py

```
# Import Libraries
import joblib
import numpy as np
import pandas as pd
pd.set_option('display.max_columns', None)
import streamlit as st
from io import BytesIO
from pathlib import Path
from DataPreprocessing import dataPreprocess
import sys
from streamlit import cli as stcli

=====
# Set to wide mode
st.set_page_config(layout="wide")
=====
# Define Variables
col_list = ['RACE', 'BMI', 'MARITAL_STATUS', 'STATE', 'RISK_TERM',
'RSK_SUM_ASSURE', 'PAYMENT_MODE', 'PAYMENT_METHOD', 'POL HOLDER_TYPE',
'SELL_AGENT_AGE', 'SELL_AGENT_RACE', 'SELL_AGENT_EDU_LEVEL', 'POLICY_NO']
=====
# Load the model in cache
```

```

@st.experimental_memo(ttl=60*5,max_entries=20)
def load_model():
    return joblib.load(open(r'XGB_ECTL_60TRH(17-11-2022).pkl', 'rb'))

load_clf = load_model()

=====
# User input - Policy No
def policyNo():
    policyNum = st.sidebar.number_input('Policy Number',
min_value=0,max_value=999999999999999, step=1)
    st.write("Policy Number : " + str(policyNum))
=====
# Collects user input features into dataframe
def userInputFeatures():

    ## Policy Holder info
    st.sidebar.write('**Policy Holder**')
    race = st.sidebar.selectbox('Race', ('Chinese', 'Indian', 'Malay',
'Bumiputera', 'Other'))
    bmi = st.sidebar.number_input('BMI')
    marital_status = st.sidebar.selectbox('Marital Status',
('Single', 'Married', 'Widowed',
'Divorced', 'Not Disclosed', 'Unknown'))
    state = st.sidebar.selectbox('State',
('KEDAH', 'KELANTAN', 'JOHOR', 'KUALA LUMPUR',
'LABUAN', 'MELAKA', 'NEGERI SEMBILAN',
'PAHANG', 'PERLIS', 'PENANG', 'PERAK',
'PUTRAJAYA', 'SABAH', 'SARAWAK', 'SELANGOR', 'TERENGGANU'))

    ## Medical Card Info
    st.sidebar.markdown("""---""")
    st.sidebar.write('**Product**')
    risk_term = st.sidebar.slider('Risk Term', 1, 100)
    risk_sum_assured = st.sidebar.selectbox('Risk Sum Assured', ('100', '150',
'200', '300', '400', '600'))
    payment_mode = st.sidebar.selectbox('Payment Mode', ('Monthly', 'Yearly',
'Half-yearly', 'Quarterly'))
    payment_method = st.sidebar.selectbox('Payment Method', ('Credit Card',

```

```

'BIRO Billing', 'Direct Selling (Cash)', 'Direct Debit', 'Group Billing',
'e-Mandate', 'Police Koperasi'))
    policy_holder_type = st.sidebar.selectbox('Policy Holder Type', ('AG',
'BC', 'FA', 'ST'))
    st.sidebar.markdown("""---""")

    ## Agent Info
    st.sidebar.write('**Agent**')
    sell_agent_age = st.sidebar.slider('Selling Agent Age', 1, 100)
    sell_agent_race = st.sidebar.selectbox('Selling Agent Race', ('Chinese',
'Indian', 'Malay', 'Bumiputera', 'Other'))
    sell_agent_edu = st.sidebar.selectbox('Selling Agent Education Level',
('Primary', 'Secondary', 'A-level', 'Diploma', 'Degree'))

    data = {'RACE': race, 'BMI': bmi, 'MARITAL_STATUS': marital_status,
'STATE': state,
            'RISK_TERM': risk_term, 'RSK_SUM_ASSURE': risk_sum_assured,
'PAYMENT_MODE': payment_mode, 'PAYMENT_METHOD': payment_method,
'POL HOLDER TYPE': policy_holder_type,
            'SELL_AGENT_AGE': sell_agent_age, 'SELL_AGENT_RACE':
sell_agent_race, 'SELL_AGENT_EDU_LEVEL': sell_agent_edu}

    return pd.DataFrame(data, index=[0])
=====
# Export prediction result to a excel file
def to_excel(df):
    output = BytesIO()
    writer = pd.ExcelWriter(output, engine='xlsxwriter')
    df.to_excel(writer, index=False, sheet_name='Sheet1')
    workbook = writer.book
    worksheet = writer.sheets['Sheet1']
    format1 = workbook.add_format({'num_format': '0.00'})
    worksheet.set_column('A:A', None, format1)
    writer.save()
    processed_data = output.getvalue()
    return processed_data
=====
# Rearrange dataframe columns

```

```

def reindexDataFrame(df):
    columns_name = ['RACE', 'BMI', 'MARITAL_STATUS', 'STATE', 'RISK_TERM',
'RSK_SUM_ASSURE', 'PAYMENT_MODE', 'PAYMENT_METHOD', 'POL HOLDER_TYPE',
'SELL_AGENT_AGE', 'SELL_AGENT_RACE', 'SELL_AGENT_EDU_LEVEL']
    return df.reindex(columns=columns_name)

def reindexDataFrame2(df):
    columns_name = ['RISK_TERM', 'BMI', 'SELL_AGENT_AGE', 'RSK_SUM_ASSURE',
'RACE_CH', 'RACE_IN',
                    'SELL_AGENT_RACE_CH', 'SELL_AGENT_RACE_IN',
'MARITAL_STATUS', 'POL HOLDER_TYPE',
                    'PAYMENT_MODE', 'PAYMENT_METHOD', 'SELL_AGENT_EDU_LEVEL',
'STATE']
    return df.reindex(columns=columns_name)
=====
def main():
    # Header
    st.write('# Early Claim and Lapsed Prediction')
    # Header of sidebar
    st.sidebar.header('User Input')

    target = pd.DataFrame()
    # Import file or user manually inputs
    uploaded_file = st.sidebar.file_uploader("Upload your input CSV file",
type=['csv', 'xlsx'])

    if uploaded_file is not None:
        if Path(uploaded_file.name).suffix == '.xlsx':
            df = pd.read_excel(uploaded_file, index_col=False)
        else:
            df = pd.read_csv(uploaded_file, index_col=False)

        # Drop useless columns
        df.drop([col for col in df.columns if col not in col_list], axis=1,
inplace=True)
        df.reset_index(inplace=True, drop=True)

        pol_df = df[['POLICY_NO']].astype('object')

```

```

df.drop('POLICY_NO', inplace=True, axis=1)

# Clean Missing Values
df = DataManipulation.missingValues(df)

df.reset_index(inplace=True, drop=True)
pol_df.reset_index(inplace=True, drop=True)

else:
    policyNo()
    df = userInputFeatures()

# Reindex the sequence of the column
dis_df = reindexDataFrame(df)

# Data Preprocessing
df = dataPreprocess(df)
df = reindexDataFrame2(df)

# Displays the user input features
st.subheader('User Input')

if uploaded_file is not None:
    dis_pol_df = pd.concat([pol_df, dis_df], axis=1)
    #st.table(dis_pol_df)
    st.dataframe(dis_pol_df, 1500, 500)
else:
    st.table(dis_df)

# Apply model to make predictions
prediction = load_clf.predict(df)
prediction_proba = load_clf.predict_proba(df)

# Prediction
st.markdown("""---""")
st.subheader('Prediction')
st.write('##### Notes: ')
st.write('##### No : Not Early Claim and Lapsed')

```

```

st.write('##### Yes: Early Claim and Lapsed if Probability(%) more than or
equal to 80%')
ecal = np.array(['Not Early Claim and Lapsed', 'Early Claim and Lapsed'])
ecal_df = pd.DataFrame(ecal[prediction], columns=['Prediction'])

proba_df = pd.DataFrame({'Probability(%)': prediction_proba[:, 1] * 100})

try:
    df2 = pd.concat([pol_df, ecal_df, proba_df], axis=1)
except UnboundLocalError:
    df2 = pd.concat([ecal_df, proba_df], axis=1)

df2 = df2.sort_values(by=['Probability(%)'], ascending=False)
st.dataframe(df2.style.format({'Probability(%)': '{:.2f}'}))

try:
    df2['POLICY_NO'] = df2['POLICY_NO'].astype(np.int64)
    df2['Probability(%)'] = df2['Probability(%)'].round(decimals=2)
    df2_xlsx = to_excel(df2)
    st.download_button(label='Export Current Result',
                        data=df2_xlsx,

file_name=f'{Path(uploaded_file.name).stem}_result.xlsx')
except KeyError:
    df2['Probability(%)'] = df2['Probability(%)'].round(decimals=2)

    df2_xlsx = to_excel(df2)
    st.download_button(label='Export Current Result',
                        data=df2_xlsx,
                        file_name=f'result.xlsx')

=====
if __name__ == '__main__':
    if st._is_running_with_streamlit:
        main()
    else:
        sys.argv = ['streamlit', 'run', 'app.py']
        sys.exit(stcli.main())

```

8.1.3 DataPreprocessing.py

```
def encode_state(data):
    data = data.str.upper()
    local_list = ['KUALA LUMPUR', 'PUTRAJAYA', 'LABUAN', 'PERLIS', 'KEDAH',
                  'PENANG', 'KELANTAN', 'TERENGGANU', 'PAHANG', 'PERAK', 'SELANGOR', 'NEGERI
SEMBILAN', 'MELAKA', 'JOHOR', 'SABAH', 'SARAWAK']
    data = data.apply(lambda x: 'OTHER' if x not in local_list else x)

    freq_dict = {
        'JOHOR': 0.189888,
        'KEDAH': 0.059074,
        'KELANTAN': 0.009899,
        'KUALA LUMPUR': 0.025386,
        'LABUAN': 0.000373,
        'MELAKA': 0.081107,
        'NEGERI SEMBILAN': 0.065407,
        'OTHER': 0.000692,
        'PAHANG': 0.020064,
        'PENANG': 0.164183,
        'PERAK': 0.138797,
        'PERLIS': 0.001011,
        'PUTRAJAYA': 0.000426,
        'SABAH': 0.004577,
        'SARAWAK': 0.104151,
        'SELANGOR': 0.134007,
        'TERENGGANU': 0.000958
    }
    data = data.map(freq_dict)
    return data

def encode_policyHolderType(data):
    freq_dict = {
        'AG': 0.980788,
        'BC': 0.011815,
        'FA': 0.001543,
        'ST': 0.005854
    }
```

```

data = data.str.upper()
data = data.map(freq_dict)
return data

def encode_paymentMode(data):
    freq_dict = {
        '1': 0.125492,
        '2': 0.040820,
        '4': 0.034380,
        '12': 0.799308
    }
    data = data.astype(str).str.upper()
    data = data.replace({'MONTHLY': '12', 'YEARLY': '1', 'HALF-YEARLY': '2',
'QUARTERLY': '4'})
    data = data.map(freq_dict)
    return data

def encode_paymentMethod(data):
    freq_dict = {
        'B': 0.005269,
        'C': 0.249548,
        'D': 0.080096,
        'F': 0,
        'G': 0.000053,
        'K': 0,
        'R': 0.665035
    }
    data = data.astype(str).str.upper()
    data = data.replace({'BIRO BILLING': 'B', 'DIRECT SELLING (CASH)': 'C',
'DIRECT DEBIT': 'D', 'GROUP BILLING': 'G',
'CREDIT CARD': 'R', 'E-MANDATE': 'F', 'POLICE
KOPERASI': 'K'})
    data = data.map(freq_dict)
    return data

def encode_marital(data):
    freq_dict = {
        'D': 0.010963,

```

```

        'M': 0.473071,
        'N': 0.015753,
        'S': 0.444332,
        'W': 0.015008,
        'X': 0.040873
    }
    data = data.astype(str).str.upper()
    data = data.replace({'DIVORCED': 'D', 'MARRIED': 'M', 'NOT DISCLOSED': 'N',
'SINGLE': 'S',
                        'WIDOWED': 'W', 'UNKNOWN': 'X'})
    data = data.map(freq_dict)
    return data

def encode_edu(data):
    freq_dict = {
        'H': 0.000426,
        'P': 0.006759,
        'S': 0.579510,
        'T': 0.376583,
        'U': 0.036722
    }
    data = data.astype(str).str.upper()
    data = data.replace({'SECONDARY': 'S', 'DEGREE': 'U', 'PRIMARY': 'P',
'A-LEVEL': 'H',
                        'DIPLOMA': 'T'})
    data = data.map(freq_dict)
    return data

def MinMaxScaler(x, min, max):
    res = (x-min) / (max-min)
    return res

def encode_riskTerm(data):
    min, max = 9, 98
    data = MinMaxScaler(data, min, max)
    return data

def encode_BMI(data):

```

```

min, max = 10.6, 32.8
data = MinMaxScaler(data, min, max)
return data

def encode_agentAge(data):
    min, max = 9, 71
    data = MinMaxScaler(data, min, max)
    return data

def encode_rskSumAssure(data):
    min, max = 100, 600
    data = data.astype(int)
    data = MinMaxScaler(data, min, max)
    return data

def encode_RaceCh(data):
    data = data.str.upper()
    data = data.apply(lambda x:1 if x == 'CHINESE' or x == 'CH' else 0)
    return data

def encode_RaceIn(data):
    data = data.str.upper()
    data = data.apply(lambda x:1 if x == 'INDIAN' or x == 'IN' else 0)
    return data

def dataPreprocess(data):
    ## Frequency Encoding
    data['POL HOLDER TYPE'] = encode_policyHolderType(data['POL HOLDER TYPE'])
    data['PAYMENT_MODE'] = encode_paymentMode(data['PAYMENT_MODE'])
    data['PAYMENT_METHOD'] = encode_paymentMethod(data['PAYMENT_METHOD'])
    data['MARITAL_STATUS'] = encode_marital(data['MARITAL_STATUS'])
    data['SELL_AGENT_EDU_LEVEL'] = encode_edu(data['SELL_AGENT_EDU_LEVEL'])
    data['STATE'] = encode_state(data['STATE'])

    ## Min Max Scale
    data['RISK_TERM'] = encode_riskTerm(data['RISK_TERM'])
    data['BMI'] = encode_BMI(data['BMI'])
    data['SELL_AGENT_AGE'] = encode_agentAge(data['SELL_AGENT_AGE'])

```

```

data['RSK_SUM_ASSURE'] = encode_rskSumAssure(data['RSK_SUM_ASSURE'])

## One Hot Encoding Encoding
data['RACE_IN'] = encode_RaceIn(data['RACE'])
data['RACE_CH'] = encode_RaceCh(data['RACE'])
data = data.drop(columns = ['RACE'])

data['SELL_AGENT_RACE_IN'] = encode_RaceIn(data['SELL_AGENT_RACE'])
data['SELL_AGENT_RACE_CH'] = encode_RaceCh(data['SELL_AGENT_RACE'])
data = data.drop(columns = ['SELL_AGENT_RACE'])

return data

```

8.2 Docker

Docker is an open-source containerization platform that enables developers to package the application with all its dependencies into a docker image. Docker ensures the environment consistency by bundling everything needed into a docker image and hence eliminates the problem where the application worked on the developer's machine but others do not. Thus, the application can run quickly and reliably from one computing environment to another with the aid of Docker. The user does not need to install anything except Docker. The steps to run the web application from Docker are described in the following section.

8.2.1 Docker Web Application Installation Steps

Data Scientist/ Developer

1. Download Docker Desktop
2. Build Docker Image for the machine learning python program with “docker build -t [app name]:latest -f Dockerfile .”
3. Build container with Docker run command “docker run --name [give a name to container] -d -p 8501:8501 [image id]” *Note: run “docker images” to check the image id.
4. Export the Image as tar file with “docker save -o early_claim_lapsed_prediction_app_image.tar.gz early_claim_lapsed_prediction_app:latest” ([How to Share Docker Images With Others, docker save](#))

IT Team/ User

5. Download Docker Desktop
6. Open a terminal/ Command Prompt
7. Load the image with "docker load < early_claim_lapsed_prediction_app_image.tar.gz"
8. Build a new container with docker run "docker run --name [container_name] -d --restart unless-stopped -p [host_port]:[container_port] [docker_image]". E.g. "docker run --name [give a name to container] -d --restart unless-stopped -p 8501:8501 [image id]". More info : ([How to Use Docker Run Command with Examples](#)) ([docker run](#)).
9. Start the container
 - a. OPTION 1: Run Start command "docker start [container name]"
 - b. OPTION 2: Go to Docker Desktop > Container, you should see the container created in the previous step. Click the Start button and click the "open with browser" button. The application will be open on the browser automatically.
10. You can visit the application via <http://localhost:8501/> or [http://\[Host IPv4 Address\]:8501/](http://[Host IPv4 Address]:8501/)

8.2.2 Docker Terminologies

1. **Docker Desktop** - An easy-to-install, lightweight Docker development environment. Available for Windows, Linux and Mac. Docker Desktop includes Docker Engine, Docker CLI client, Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper.
2. **Docker Image** - A read-only template that contains a set of instructions for creating a container that can run on the Docker platform.
3. **Docker Container** - A container is a runnable instance of an image. Is an isolated, standalone, executable software package which includes application and its dependencies (e.g. python libraries and configuration). Well-suited for securely running software like databases or web applications that need access to sensitive resources without giving access to every user on the system
4. **Dockerfile** - A Dockerfile contains all the code to set up a docker container from downloading the docker image to set the environment. You can think of it as describing the complete operating system installation of the system that you want to run.

8.2.3 Dockerfile

```
FROM python:3.8

WORKDIR /app

COPY requirements.txt ./requirements.txt

RUN pip install --upgrade pip
RUN pip install -r requirements.txt

EXPOSE 8501

COPY . /app

ENTRYPOINT [ "streamlit","run" ]

CMD [ "app.py" ]
```